Linux Advanced

m2iformation.fr

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Linux Advanced

Table of Contents

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Linux System Architecture

Linux System Architecture

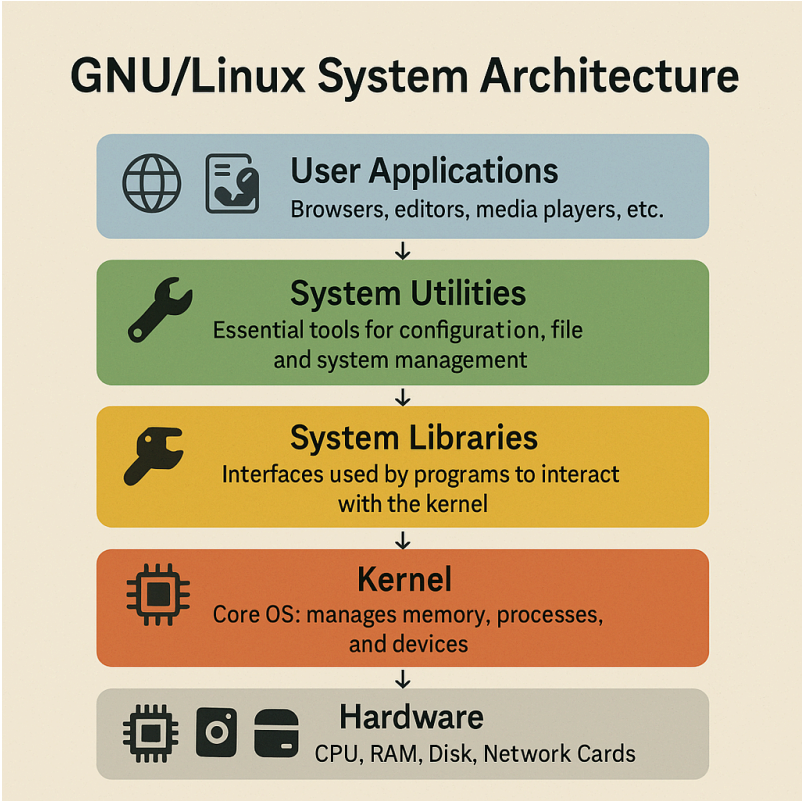
Introduction

- The Linux system is structured according to a modular architecture with distinct layers, including hardware, a kernel, and software layers that interact through standardized interfaces.
- This architecture ensures security, stability, and modularity.

Linux System Architecture

Overview of the Linux Architecture

- **Hardware**: CPU, memory, input/output devices, hard drives, etc.
- **Linux Kernel**: the core layer responsible for managing hardware resources.
- **Kernel Modules (LKM)**: dynamic kernel extensions.
- **Shell and System Utilities**: allowing users to interact with the system.
- **System Libraries (glibc, libm, libpthread, etc.)**: provide hardware abstraction.
- **User Applications**: software executed by users.



Linux System Architecture

Overview of the Linux Architecture

- The Linux system is structured in layers:
  - at the bottom, the **hardware** (CPU, memory, peripherals);
  - above it, the **kernel** which runs in privileged mode;
  - and at the top, the **user space** where applications and user processes run.
- This separation between kernel space and user space is fundamental for stability and security: the kernel has direct access to the hardware and manages resources, while user programs must go through controlled requests (system calls) to access its services.
- This provides memory protection and prevents a user-space program from directly corrupting the kernel or other processes.

Linux System Architecture

Overview of the Linux Architecture

- In practice, when an application wants to access the disk, the network, or any peripheral device, it makes a system call to ask the kernel to perform this privileged operation.
- The kernel thus acts as an intermediary between hardware and software:
  - it manages **memory**,
  - schedules **processes**,
  - provides abstractions such as **file systems** and **network interfaces**,
  - and exposes these features to applications through a well-defined interface.

Linux System Architecture

Protection Rings (-1, 0, 3)

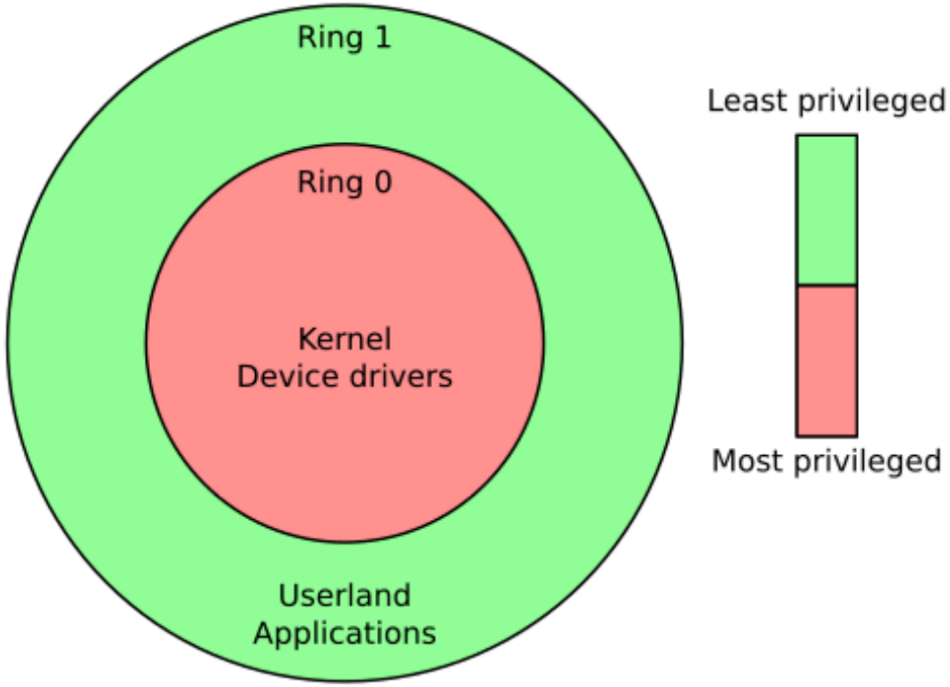
Linux leverages protection rings to ensure effective isolation between processes and the kernel:

Ring	Level	Description
-1	Hypervisor	Hardware virtualization and firmware (BIOS/UEFI)
0	Kernel mode	Full access to hardware, critical operations, process and hardware management
1	Not used	Not used by Linux to simplify the architecture
2	Not used	Not used by Linux to simplify the architecture
3	User mode	Execution of applications and libraries. Limited and controlled access to hardware via system calls

Thus, user applications have no direct access to hardware, which ensures security and stability.

Linux System Architecture

Protection Rings (-1, 0, 3)



Linux System Architecture

Protection Rings (-1, 0, 3)

- On x86 processors, the protection mode using **rings** defines several levels of hardware privilege.
- The most privileged ring is **ring 0**, used by the kernel (supervisor mode), and the least privileged is typically **ring 3**, used by user-space applications (user mode).
- In theory, x86 offers 4 rings (0 to 3), but most operating systems (Unix, Windows...) only use two: ring 0 for the kernel and ring 3 for user space.
- Linux follows this model, leaving the intermediate rings unused.
- This separation ensures that a program running in ring 3 cannot execute sensitive instructions or directly access hardware without going through ring 0 — any unauthorized attempt triggers a general protection fault, which is intercepted by the kernel.

Linux System Architecture

Protection Rings (-1, 0, 3)

- Linux follows this model, leaving the intermediate rings unused.
- This separation ensures that a program running in ring 3 cannot execute sensitive instructions or directly access hardware without going through ring 0 — any unauthorized attempt will trigger a general protection fault intercepted by the kernel.

Linux System Architecture

Protection Rings (-1, 0, 3)

- **Hypervisors** (virtualization) make use of an even higher privilege level, often referred to as **ring -1**.
- Hardware virtualization extensions (Intel VT-x, AMD-V) introduce a special execution mode for the hypervisor, more privileged than the kernel itself.
- In this mode, the hypervisor (such as KVM or Xen) runs in *ring -1* and has full control over the machine, while each guest operating system runs in a virtual ring 0 under its supervision.
- Before the advent of these extensions, hypervisors had to rely on clever techniques: for example, Xen would run guest kernels in ring 1 and intercept unauthorized privileged instructions using a *trap-and-emulate* mechanism.
- Today, thanks to VT-x *"root mode"*, the hypervisor operates in a dedicated level (ring -1) to manage access, allowing guest OSes to run in ring 0 without compromising isolation.
- In summary, **ring 0** corresponds to the Linux kernel (and drivers) running in supervisor mode, **ring 3** to user-space programs running in unprivileged mode, and **ring -1** to the hypervisor mode enabled by hardware virtualization for hosting virtual machines with a higher privilege level than the host kernel.

Linux System Architecture

Hardware Platforms Supported by Linux

- One of the great strengths of the Linux kernel is its **portability**: it has been ported to a very wide range of hardware architectures.
- Originally developed on the Intel 80386 PC, Linux now supports **dozens of different architectures**.

Linux System Architecture

Hardware Platforms Supported by Linux

- **x86 32-bit (IA-32)** and **x86 64-bit (x86_64/AMD64)** – mainstream PC, server, and laptop architectures.
- **ARM** (32-bit ARMv7 and 64-bit ARMv8/AArch64) – ubiquitous in embedded systems, smartphones, tablets, and single-board computers (Raspberry Pi, etc.).
- **RISC-V** – an open and modular RISC architecture, officially supported in the Linux kernel since 2022 (RISC-V - Wikipedia).
- **PowerPC/POWER** – IBM's RISC architecture used in workstations, servers, and older Macs (Linux runs on IBM Power systems and older gaming consoles like the PS3, based on PowerPC).
- **MIPS** – a RISC architecture formerly widespread in embedded systems and SGI workstations, supported by Linux.
- **SPARC** – Sun/Oracle's RISC architecture (Unix workstations), also supported by Linux.
- **IBM S/390 (s390x)** – the architecture of IBM mainframes, natively supported by Linux since the early 2000s.
- Other platforms: **Microblaze**, **MIPS64**, **SuperH**, **Alpha**, **PA-RISC**, **Itanium**, **ARC**, etc.

Linux System Architecture

Hardware Platforms Supported by Linux

- For example, a distribution like **Ubuntu** provides official images for x86_64, ARM64, PowerPC64, and IBM System z, illustrating the wide range of supported hardware.
- Linux can run just as well on a **microcontroller** clocked at a few MHz with limited memory as on a massive **supercomputer** — showcasing its exceptional adaptability.
- This portability is made possible by the kernel's abstraction of hardware: a large portion of the code is architecture-independent, and only a few layers (CPU management, interrupts, etc.) are architecture-specific, often isolated in subdirectories of the source code (e.g., `arch/x86`, `arch/arm`...).
- Supporting a new architecture involves implementing these adaptation layers.

Linux System Architecture

Linux Kernel and LKM (Loadable Kernel Modules)

Linux Kernel

- The Linux kernel follows a **modular monolithic architecture**. **Monolithic** means that the kernel is a single block of code running in kernel space (as opposed to a microkernel, which would split services into separate processes).
- **Modular** means that this kernel can be extended on the fly through **loadable modules**.
- Indeed, Linux is designed in a modular way, allowing kernel components to be integrated as software modules that can be dynamically loaded or unloaded as needed.
- When compiling the kernel, many drivers and features can be selected either as built-in or as separate modules (`.ko` files).

Linux System Architecture

Linux Kernel and LKM (Loadable Kernel Modules)

Linux Kernel

The kernel is the core of the Linux system, responsible for:

- **Managing hardware resources** (CPU, RAM, I/O, devices...).
- **Multitasking and scheduling** (scheduler).
- **Memory management** (allocation, paging, virtual memory...).
- **File system and storage** (FS).
- **Device management**.
- **Security and access control** (permissions, isolation, security).
- **Inter-process communication** (IPC).
- **Networking and associated protocols**.

Linux System Architecture

Linux Kernel and LKM (Loadable Kernel Modules)

LKM (Loadable Kernel Modules)

Linux Kernel Modules (LKM) are software components that can be dynamically loaded or unloaded to:

- Extend the kernel's functionalities without rebooting (hardware drivers, network protocols, file systems).
- Simplify system maintenance (bug fixes, enhanced security).
- Optimize system resources by loading only the necessary modules.

Linux System Architecture

Linux Kernel and LKM (Loadable Kernel Modules)

LKM (Loadable Kernel Modules)

- **LKM (Loadable Kernel Modules)** are kernel modules that can be inserted or removed at runtime. They offer several advantages:
  - adding a **module** enables a new feature (e.g., support for a new file system or device) without recompiling or rebooting the kernel;
  - removing a module frees up the associated resources if the hardware is no longer in use. This helps reduce the kernel's memory footprint (only necessary components are loaded) and makes updates easier (a module can be replaced with a new version without shutting down the entire system).
- For example, drivers for some network cards, printers, or file systems (e.g., **ext4**, **XFS**, **NTFS**...) are often provided as modules that can be loaded when needed.

Linux System Architecture

Linux Kernel and LKM (Loadable Kernel Modules)

LKM (Loadable Kernel Modules)

- A **module** is a separately compiled piece of code (a `.ko` file) that can be inserted into kernel space.
- The **kernel** maintains a symbol table of exported symbols that modules can reference. When a module is loaded (using `insmod` or `modprobe`), the kernel performs hot linking: it resolves the module's dependencies on kernel symbols or other already-loaded modules, allocates kernel memory for the module, and transfers execution to it.
- Once loaded, the **module** runs with the same privileges as the kernel itself.
- Linux provides internal APIs for writing **modules**, for instance to register a new driver, a network protocol, etc., through initialization functions that are called upon loading.
- Conversely, when a module is removed (`rmmod`), the kernel calls the module's cleanup routine and then frees its resources.
- It is important to note that a poorly written (buggy) **module** can potentially crash the system just like a bug in the monolithic kernel, since it operates at the same privilege level.

Linux System Architecture

Linux Kernel and LKM (Loadable Kernel Modules)

In Summary

Linux combines the best of both worlds:

- a high-performance **monolithic kernel** where all code runs in supervisor mode, and **modular extensibility** that allows great flexibility in usage.
- Most distributions ship with a generic kernel that includes a minimal set of built-in features, with everything else provided as modules (stored under `/lib/modules/` corresponding to the kernel version).
- At boot time, only the base kernel is loaded;
- then, the necessary modules (drivers, etc.) are inserted on the fly (often automatically via `udev` or *startup scripts*) based on the detected hardware or system needs.

Linux System Architecture

Linux Kernel and LKM (Loadable Kernel Modules)

Key commands for module management:

```
lsmod              # Displays currently loaded modules
modprobe <module> # Loads a specific module
rmmod <module>     # Unloads a module
insmod <module>    # Directly loads a module (.ko)
modinfo <module>   # Displays information about a module
```

Example of managing a network module:

```
sudo modprobe e1000e        # load module for Intel network card
sudo rmmod e1000e           # unload module
```

Linux System Architecture

The Root Filesystem (`/`)

- On Linux, the entire file system is organized around a single root called `/` (the *root filesystem*).
- This hierarchy follows the **FHS (Filesystem Hierarchy Standard)**, a standard that consistently defines the locations of files and directories on Unix-like systems.

Linux System Architecture

The Root Filesystem (`/`)

The root filesystem is the hierarchical base of a Linux system, from which all other filesystems are mounted.

Typical structure:

```
/
├── /bin      (essential binaries)
├── /boot     (boot-related files)
├── /etc      (system configuration files)
├── /home     (user directories)
├── /var      (variable data, logs, databases)
├── /lib      (essential shared libraries)
├── /usr      (applications, libraries, resources)
├── /bin, /sbin (core system commands)
├── /tmp      (temporary files)
└── /dev      (hardware devices)
```

Linux System Architecture

The Root Filesystem (`/`)

- This standardized organization (defined by FHS) allows both administrators and users to easily navigate the system, and enables software to know where to place or look for files.
- For example, a program can assume that system configurations are in `/etc`, logs are in `/var/log`, and executables are in `/usr/bin` or `/usr/local/bin` depending on whether they are system-wide or locally installed.
- Most Linux distributions strictly follow the FHS, with a few slight variations or symbolic links (e.g., some distributions unify `/bin` and `/usr/bin` by making one a symlink to the other to simplify the hierarchy).
- Overall, a user switching between distributions will find the same basic structure, inherited from Unix.

Linux System Architecture

Device Drivers

- **Device drivers** are kernel components responsible for communicating with hardware or virtual devices.
- Their role is to provide a standard interface to programs for using a given piece of hardware, handling all the low-level, device-specific details.
- In Linux, almost everything is managed by drivers: disks, USB keys, network cards, graphics cards, sound devices, as well as more virtual elements like the `proc` pseudo-filesystem or virtual terminals.

Linux System Architecture

Device Drivers

From the kernel's perspective, a driver typically registers as a specific **type of device**:

- **Character drivers** – manage devices accessed as a stream of bytes, one at a time. Typical examples include serial ports, terminals, input devices (keyboards), or `/dev/tty`, `/dev/random`, etc. Operations use system calls like `read()`/`write()` to transfer bytes sequentially.
- **Block drivers** – manage devices organized in addressable blocks (typically 512 bytes or more). These mainly include **disks** and other storage media (SSDs, CD-ROMs). The kernel uses them through the block cache and system calls like `read()/write()` in blocks, allowing the mounting of file systems.
- **Network drivers** – unlike the others, they are not exposed as files in `/dev` but instead integrate into the kernel's network stack. Their interface is more complex: they transmit **frames** or network packets instead of simple bytes or blocks. Examples include Ethernet or Wi-Fi drivers (packet transmission/reception) and the loopback interface. Access is handled through the user-space socket/Berkeley API, which internally relies on these network drivers.

Linux System Architecture

Device Drivers

- In practice, Linux drivers can either be **statically built into the kernel** or compiled as **loadable modules** (LKM), as previously discussed.
- Most distributions choose to ship the majority of drivers as modules, so they are only loaded if the corresponding hardware is present.
- When the system detects new hardware (e.g., a USB key is plugged in), a mechanism like **udev** may automatically load the appropriate kernel module to handle it.
- Conversely, an unused driver module can be unloaded to free memory.
- You can list currently loaded drivers (modules) using the `lsmod` command, which displays a list of kernel modules currently in memory.
- For example, you might see entries like `usb_storage`, `i915` (Intel GPU driver), etc., along with their size and usage count.
- Technically, `lsmod` simply formats and displays the contents of `/proc/modules`.

Linux System Architecture

Device Drivers

In Summary:

- Device drivers are kernel code that interfaces with hardware.
- They often expose an abstraction (e.g., a file in `/dev` or a network interface) that programs can use through standard system calls.
- Thanks to drivers, applications don't need to know the specifics of how a network card or disk works: they use generic calls (`open`, `ioctl`, `read`, `write`...), and the driver, behind the scenes, translates those requests into actual hardware operations.

Linux System Architecture

Device Drivers

In Summary:

- The quality and breadth of Linux's driver ecosystem is one of its greatest strengths: today, the kernel supports an immense range of hardware.
- Useful commands for interacting with drivers include:
  - `lsmod` – list loaded modules (i.e., active drivers).
  - `modinfo <module>` – display information about a module (version, description, license, supported hardware aliases).
  - `lspci`, `lsusb` – list connected PCI/USB devices (to identify available hardware and determine associated modules).
  - Files in `/proc` or `/sys` – e.g., `/proc/interrupts` to see which drivers are using which IRQs, `/sys/bus/usb/devices/.../driver` to see which driver is handling a specific USB device, etc.

Linux System Architecture

Shared and Static Libraries

- **Libraries** are collections of functions/utilities shared by multiple programs.
- On Linux (and Unix in general), there are two main modes of linking to libraries: **static linking** and **dynamic (shared) linking**.

Linux System Architecture

Shared and Static Libraries

- **Static libraries**:
  - These are `.a` (archive) files containing object code.
  - When compiling/linking a program, you can statically link certain libraries — the necessary code from the library is then copied **into the final executable**.
  - The advantage is that the executable has no external dependencies at runtime (it is self-contained), but the downside is a larger file size and duplication of the same code in memory if multiple programs use the same library.

Linux System Architecture

Shared and Static Libraries

- **Dynamic/Shared libraries**:
  - These are `.so` (*shared object*) files.
  - In this case, the executable does **not** embed the library code;
  - it only contains a reference stating "I will need this shared library."
  - At runtime, the system **loads** the shared library into memory (only once, even if multiple processes use it) and links it to the program.
  - This way, multiple applications can **share** the same library in memory, significantly reducing overall footprint.
  - Furthermore, updates are centralized: fixing a bug in the library immediately benefits all programs that use it (no need to recompile each one).
  - These reasons — **reduced disk size**, **lower memory usage**, and **centralized updates** — have made dynamic linking the dominant approach in modern systems.

Linux System Architecture

Shared and Static Libraries

- On Linux, almost all applications use shared libraries (starting with **glibc**, the standard C library, which provides system calls, memory management, standard functions, etc.).
- When a program is launched, the **dynamic linker** (`ld-linux.so`, also known as the *runtime loader*) is responsible for locating and loading the required `.so` files into memory.
- It searches in standard paths (`/lib`, `/usr/lib`...), and optionally in the `LD_LIBRARY_PATH` environment variable or the cache file `/etc/ld.so.cache` generated by `ldconfig`.
- If a required library is missing, the program will fail to start (error: *"libXYZ.so.1 not found"*).

Linux System Architecture

Shared and Static Libraries

- To find out which shared libraries an executable depends on, use the **`ldd`** command (short for **l**i**d** Dynamic Dependencies).
- For example, `ldd /bin/ls` will list all the libraries required by `/bin/ls` (libc, libpthread, libselinux, etc.) and where they are located on the system.
- Internally, `ldd` works by running the dynamic loader in a special mode (using `LD_TRACE_LOADED_OBJECTS`) to display this information without actually executing the program.
- It's a very useful tool for diagnosing missing dependencies or simply understanding what libraries a binary uses.
- Another valuable tool is **`strace`**, which traces the system calls made by a program.
- Although strace isn't specific to libraries, it can be used to observe the loading process: when launching an application with `strace`, you'll see `open("/lib/xyz.so", ...)` calls searching for libraries, followed by `mmap()` calls mapping them into memory, etc. Strace reveals *the thin layer between user processes and the kernel*.
- This includes opening libraries, searching for configuration files (e.g., `/etc/ld.so.cache`), and more.
- In case of loading errors, strace can show that the process is searching for a library in a specific directory and encounters an `ENOENT` (file not found) error — helping to fix the issue (e.g., by installing the missing library or adjusting `LD_LIBRARY_PATH`).

Linux System Architecture

Shared and Static Libraries

In Summary

- Shared libraries (`.so`) are a cornerstone of Linux systems, enabling efficient **code sharing** between applications. Tools like `ldd` help audit a program's dependencies, while `strace` can dynamically observe a process's behavior (including how it loads those dependencies).
- Dynamic linking is generally recommended, except for specific use cases (e.g., ultra-portable embedded software or the need for a standalone binary with no external dependencies).
- Conversely, static linking can be useful in minimalist environments or when avoiding reliance on system libraries (at the cost of code duplication).
- Linux supports both methods, but most distributions rarely use `.a` files—some even exclude them by default, providing only `.so` files and development headers to encourage dynamic linking.

Linux System Architecture

System Calls

- System calls (or "syscalls") in Linux are the mechanism through which user-space applications interact with the kernel to request services such as hardware access or process management.
- When a program needs a service that only the kernel can provide (e.g., reading a file, allocating memory, creating a process), it performs a system call.
- A syscall resembles a function call, but it triggers a switch from **user mode to kernel mode** via a hardware trap or interrupt mechanism.

Linux System Architecture

System Calls

Here are some key points:

1. **Interface between User Space and Kernel**
   - System calls provide a standardized API allowing programs to perform critical operations without direct hardware access.
   - This ensures that sensitive operations (like file I/O, memory management, or inter-process communication) are handled securely.
2. **Invocation Mechanism**
   - On Linux, system calls are typically implemented via specific CPU instructions (e.g., `syscall` on x86_64 architectures or `int 0x80` on older versions).
   - This triggers a switch to kernel mode, allowing the kernel to execute the corresponding system call handler.

Linux System Architecture

System Calls

3. **Examples of System Calls**
   - **File Management:** `open`, `read`, `write`, `close`
   - **Process Management:** `fork`, `execve`, `waitpid`, `exit`
   - **Memory Management:** `mmap`, `munmap`
   - **Inter-Process Communication:** `pipe`, `socket`, `bind`, `listen`, `accept`
4. **Security and Stability**
   - By isolating applications from direct hardware access and requiring kernel mediation for critical operations, Linux enhances both system security and stability.
   - The kernel can enforce security policies, resource management, and access control to prevent malicious or faulty behavior.

Linux System Architecture

System Calls

In Summary

- System calls form the **interface contract** between the Linux kernel and user applications.
- They offer a set of services (process, memory, file, IPC, network management, etc.) that a program can invoke.
- This interface is relatively stable and standardized (POSIX), which makes software portable: recompiling a POSIX-compliant program on Linux will use the appropriate syscalls via `glibc`.
- System call performance is critical, which is why the kernel and CPUs offer optimizations (e.g., on x86_64, the `syscall` instruction is much faster than the older `int 0x80`).
- Linux also provides mechanisms like *vdso* (Virtual Dynamic Shared Object) to expose certain kernel routines in user space without traps (e.g., `gettimeofday` can be done without a syscall by reading a shared memory region, avoiding trap overhead).
- Still, for most operations, switching to kernel mode is necessary.
- Tools like `strace` remind us just how deeply every user-space action relies on these kernel calls — they are the safeguard separating the **restricted user world** from the **real system resources**.

Linux System Architecture

Different Shells (Bash, Zsh, Fish, etc.)

- A *shell* is the command interpreter in a Unix/Linux system.
- There are several shells, each with slightly different features and philosophies, but they all serve the same core purpose:
  - to let users execute commands,
  - write scripts,
  - and automate tasks.
- On Linux, the default shell is often **Bash** (Bourne Again Shell), but other popular shells include **Zsh** (Z Shell), **Fish** (Friendly Interactive Shell), **Tcsh**, **Ksh**, etc.

Linux System Architecture

Different Shells (Bash, Zsh, Fish, etc.)

Main Linux Shells:

- **Bash** (`/bin/bash`): the most common, standard shell
- **Sh** (Bourne Shell): old but widespread, minimalist
- **Zsh**: powerful, supports plugins, advanced autocomplete
- **Fish**: user-friendly shell, built-in visual autocompletion
- **Ksh (Korn Shell)**: robust and performant for complex scripting

Linux System Architecture

Different Shells (Bash, Zsh, Fish, etc.)

Each of these shells has its fans and use cases:

- **Bash** remains the de facto standard for scripting and system administration, thanks to its universal availability and POSIX compliance. Even users of Zsh or Fish often rely on Bash to execute `/bin/sh` scripts.
- **Zsh** is often preferred by those who want to enhance their working environment with improved prompts, smart completion, and high customizability. With the right setup, Zsh can significantly speed up complex command entry.
- **Fish** is well-suited for users who want an efficient interactive shell without complex setup, or beginners who benefit from Fish's visual aids and friendly experience.

Linux System Architecture

Different Shells (Bash, Zsh, Fish, etc.)

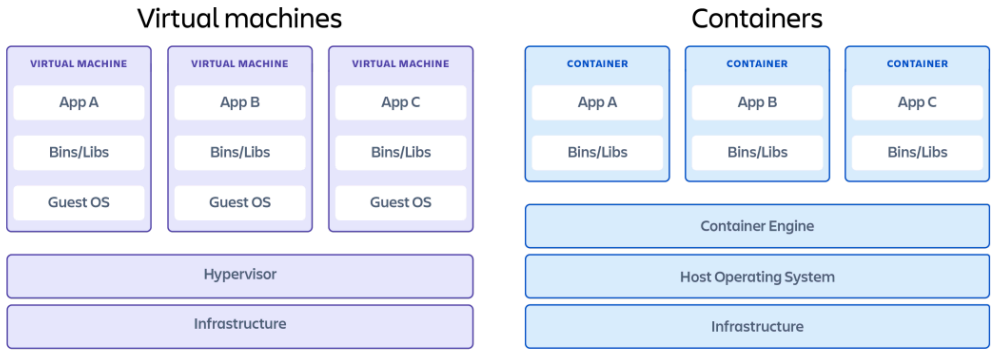
In Summary

- All these shells ultimately serve the same purpose and let users accomplish the same tasks.
- Switching from one to another is mostly a matter of comfort and habit.
- A standard Bash script can usually be run under Zsh (which is largely compatible) but not directly under Fish (which is not Bash-compatible by default).
- Some advanced users even use multiple shells depending on context (e.g., Bash for scripting, Zsh for interactive work on their PC, Fish on another system for experimentation).
- Thanks to shell abstraction, the user environment can be customized without modifying the underlying layers of the system.

Linux System Architecture

Virtualization on Linux

- In a traditional virtualization scenario, a hypervisor (or a host kernel in the case of KVM) allows multiple full guest operating systems to run — each VM has its own kernel and libraries.
- On the right, with containerization, a single host kernel is shared, and a *container engine* isolates multiple applications in separate but lightweight environments (no duplication of the full operating system).



Linux System Architecture

Virtualization on Linux

Hardware Virtualization (Ring -1)

- **Type 1 Hypervisors (bare-metal):**
  - VMware ESXi, XenServer, Proxmox VE
- **Type 2 Hypervisors (hosted):**
  - VirtualBox, VMware Workstation, KVM/QEMU

Linux System Architecture

Virtualization on Linux

Full Virtualization with Hypervisor (KVM, Xen, etc.):

Linux can serve as both a **type 1** and **type 2** hypervisor. The most notable project is **KVM (Kernel-based Virtual Machine)**, which is directly integrated into the Linux kernel.

- KVM transforms the kernel into a hypervisor:
  - it uses a kernel module (`kvm.ko`) that leverages hardware virtualization extensions (Intel VT-x, AMD-V) to create virtual machines.
  - Each KVM VM is essentially a Linux process (managed by the standard scheduler) with a separate address space for the guest OS.
  - The Linux host kernel acts as the hypervisor, managing access to CPU, memory, and devices across VMs.
  - KVM benefits from the entire Linux ecosystem (efficient scheduler, existing drivers for virtual I/O via virtio, etc.).
  - In short, KVM is considered an **in-kernel hypervisor** (some label it as type 2 because it uses a host OS, but functionally it behaves like a type 1).

Linux System Architecture

Virtualization on Linux

Quick Example with KVM:

Install KVM:

```
sudo apt install qemu-kvm libvirt-daemon bridge-utils virt-manager
```

Create a VM with KVM:

```
virt-install --name vm-test --ram 2048 --disk size=10G --os-variant ubuntu22.04 --cdrom ubuntu.iso
```

Linux System Architecture

Virtualization on Linux

- In contrast, **Xen** is a traditional type 1 hypervisor that runs directly on the hardware, independently of a host OS.
- At boot, the Xen hypervisor loads first (instead of a typical kernel) and creates a privileged environment (*dom0*) that runs Linux to control the hardware.
- Other VMs (*domU*) can run either via **paravirtualization** (the guest OS knows it's virtualized and calls Xen for certain operations, which requires OS modifications but improves performance), or **full virtualization** (with CPU extensions, Xen emulates standard hardware if the OS isn't modified, with slightly more overhead).
- Xen provides strong isolation and was widely adopted in server environments, though it is more complex to set up due to the need for a dedicated hypervisor.

Linux System Architecture

Virtualization on Linux

- Today, **KVM** has become highly popular because it is included directly in the Linux kernel (no external hypervisor required), benefits from community development, and is easier to maintain (no need for external kernel patches).
- In practice, virtualization platforms like **QEMU/KVM** (with libvirt, etc.) use **KVM** for performance and **QEMU** for hardware emulation, delivering a full experience comparable to VirtualBox or VMware — entirely on Linux.

Linux System Architecture

Virtualization on Linux

Containers (LXC, Docker, etc.):

- Rather than emulating a full machine, Linux offers **container-based isolation**, often called OS-level virtualization. This approach uses kernel features like **namespaces** and **cgroups** to isolate processes as if they were running on separate systems, while sharing the same kernel.
- Early projects like **LXC (Linux Containers)** pioneered this model, and the **Docker** ecosystem has since made it mainstream by simplifying application deployment in lightweight containers.
- In a container, the **view of processes**, **filesystem**, **network**, and **users** is isolated, so a process inside a container feels like it's alone on the system.
- However, all containers share the **host Linux kernel** — they do not run their own.
- As a result, starting 10 containers does not mean running 10 kernels, just 10 isolated groups of processes.
- This offers enormous **efficiency**: containers can start in milliseconds, use very little extra RAM, and avoid OS redundancy.

Linux System Architecture

Containers (LXC, Docker, etc.):

The trade-off with containers is that they **share the kernel**:

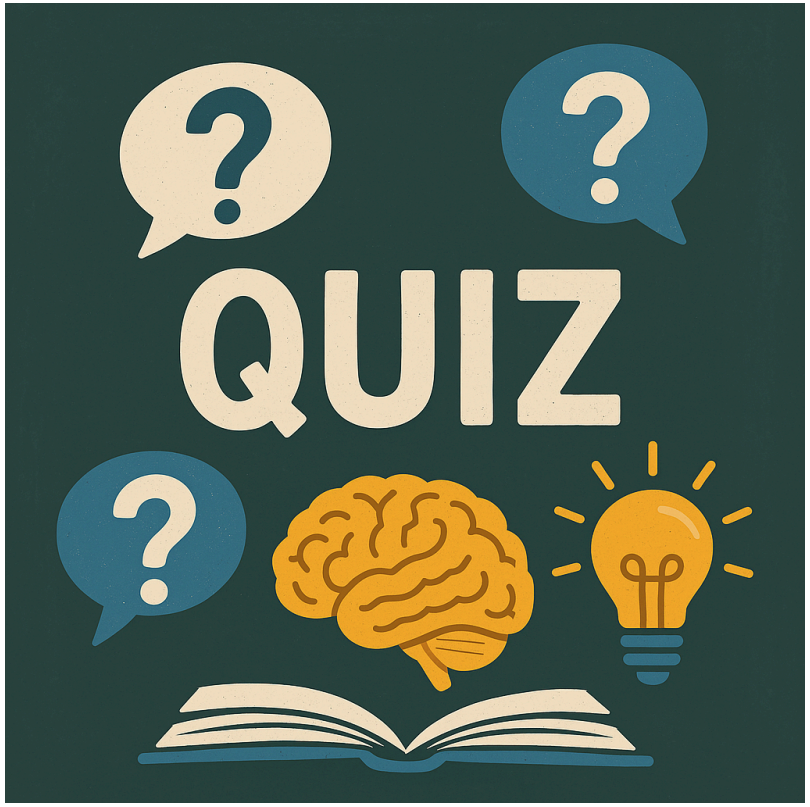
- This means you cannot run a non-Linux OS in a Linux container (e.g., you can't run Windows in a Linux container — unlike with VMs).
- Also, security depends on kernel isolation;
- if an attacker breaks out (via a kernel vulnerability), they may escape the container.
- Nevertheless, in practice, Linux containers provide sufficient isolation for many use cases and allow far higher density than VMs.
- It's a modern evolution of traditional *chroot* or *jails*:
  - each container has its own system view (private mounts, virtual networks, isolated users), while being far more **agile** than a VM.
  - Docker further enhanced this with stackable image formats and an ecosystem that revolutionized software deployment.

Linux System Architecture

Virtualization on Linux

- **Linux is at the heart of both virtualization models (heavyweight and lightweight):**
  - as a **KVM hypervisor** for running VMs in public clouds, and as a **container engine** (with Docker/Containerd) to isolate applications.
  - Thus, Linux offers a **full spectrum**: from heavyweight VMs with full OS flexibility to lightweight containers sharing the host kernel — and even intermediate solutions like LXC, which offer VM-like environments without a full hypervisor.
  - This versatility makes Linux the backbone of most modern virtualized infrastructures.
  - Linux provides full subsystems in both its documentation and codebase to support virtualization (e.g., KVM as hypervisor, cgroups/namespaces for containers, KVM API via `/dev/kvm`, `/proc` interfaces for cgroups, etc.), and continues to innovate (e.g., projects like **Kata Containers**, which combine lightweight VMs and container isolation, or **virtio-fs** for efficient shared filesystems between host and VM).

🧠 Quiz Time



UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Linux kernel

Linux Kernel

Downloading the Sources and Required Tools

- To compile the Linux kernel, you first need to download its **official source code** and install the required **compilation tools**.
- The Linux kernel source code is available on the official website **kernel.org**, which lists *mainline* (development), *stable*, and *LTS* (long-term support) versions.
- You can either clone Linus Torvalds' Git repository or download the archive of the desired stable version (`.tar.xz` files) from kernel.org (Compiling and Installing the Linux Kernel | matteyeux's blog).
- For example, to download version 6.1.8, you would use:

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.8.tar.xz
tar -xvf linux-6.1.8.tar.xz && cd linux-6.1.8
```

Linux Kernel

Downloading the Sources and Required Tools

- Another option on Debian is to **install the corresponding source package** (`apt install linux-source-version`), which places an archive of the kernel source (e.g., `/usr/src/linux-source-5.10.tar.xz`) that you simply extract into a user workspace directory.
- It is recommended to *avoid compiling as root* or directly in `/usr/src`; instead, use a directory inside your *HOME* (e.g., `~/kernel/`) to prevent permission issues or conflicts with the system.

Linux Kernel

Downloading the Sources and Required Tools

- Install the required **build tools**.
- On Debian/Ubuntu, the meta-package `build-essential` provides the C compiler (GCC) and make.
- You also need development libraries for Ncurses (for `menuconfig`) and other utilities. For example:

```
sudo apt install build-essential libncurses-dev bison flex libssl-dev libelf-dev bc
```

- This command installs GCC, Make, Ncurses libraries, Bison, Flex, OpenSSL (for cryptographic options), and libelf (used for BPF generation, etc.).
- Other distributions may use slightly different package names, but these components are generally required.
- You may also find `git` useful (if cloning the latest source) and `fakeroot` (to create packages without root privileges — useful for Debian packaging).

Linux Kernel

Downloading the Sources and Required Tools

Once the archive is extracted, you'll have a directory (e.g., `linux-6.1.8/`) containing many subdirectories.

- **The most important ones include:**
  - `arch/` – architecture-specific code (x86, ARM, etc.). For instance, `arch/x86` contains code specific to 32/64-bit PCs.
  - `drivers/` – all device drivers, organized by hardware type (sound, network, USB, etc.).
  - `fs/` – file system implementations (ext4, NTFS, FAT, etc.), with one subdirectory per supported FS.
  - `kernel/` – the core kernel logic (scheduler, timers, synchronization, etc.).
  - `mm/` – memory management (with architecture-specific parts in `arch/*/mm/`).
  - `net/` – the kernel's networking stack (protocols, sockets, etc.).
  - `include/` – shared header files (`.h`).
  - `scripts/` – helper scripts for configuration and compilation.

Linux Kernel

Advanced Kernel Configuration

- Before compiling, you must **configure the kernel** according to your hardware and functional needs.
- The configuration defines which drivers and options will be compiled (built-in or as modules).
- The classic method is to use the semi-graphical interface `make menuconfig`.

Simple example:

```
cd linux-6.8
make menuconfig
```

- Select the needed modules/drivers.
- Save and exit (a `.config` file will be automatically generated).

Linux Kernel

Advanced Kernel Configuration

The `menuconfig` interface is organized into **major thematic sections**. Key configuration areas include:

- **General setup** – general kernel options (e.g., define a **custom local version** suffix to append to the kernel version, useful to identify your build).
- **Processor type and features** – CPU and architecture-related settings (optimize for a specific CPU, enable/disable SMP, hyperthreading, etc.).
- **Power management and ACPI options** – power-saving features, suspend/resume, ACPI support (especially important for laptops).
- **Networking support** – protocol support (IPv6, IPsec, Bluetooth, etc.) and network security options.
- **Device Drivers** – manage support for various hardware: graphics cards, network adapters, storage, USB, etc. Enable or disable based on your system.
- **File systems** – support for file systems like ext4, Btrfs, NFS, etc., that you want included.
- *(And so on: security frameworks like SELinux or AppArmor, cryptography, virtualization, "Kernel hacking" options for debugging, etc.)*

Linux Kernel

Advanced Kernel Configuration

Advanced Customization:

- You can load an existing configuration as a starting point.
- A common practice is to **reuse your currently installed kernel's configuration**.
- On Debian/Ubuntu, this file is found in `/boot` (e.g., `config-5.10.0-17-amd64`).
- Copy it into your kernel source directory as `.config`:

```
cp /boot/config-$(uname -r) .config
```

- This gives you a known good configuration.
- Then run `make menuconfig` and only tweak what you need.
- If you're compiling a newer kernel version, run **`make oldconfig`** to update the configuration:
  - It will prompt you only for new options introduced since your base config (you can also run `make olddefconfig` to accept all defaults automatically).

Linux Kernel

Kernel Compilation and Installation (Classic Method)

- Once your configuration is ready, you can proceed with **compiling and installing** the kernel.
- The “classic” method uses Make directly to build the kernel and its modules, followed by a manual installation process.

Linux Kernel

Kernel Compilation and Installation (Classic Method)

Step 1: Compiling the Kernel

- Start the actual compilation with the `make` command.
- It is recommended to use the `-j` option to parallelize the build based on the number of available CPU cores (e.g., `make -j$(nproc)` uses all cores).
- From the kernel source root directory:

```
make -j$(nproc)
```

- This command will compile the kernel image (usually a compressed binary file called **bzImage**) along with all configured modules.
- Compilation time can range from a few minutes to over an hour depending on system performance and the kernel's size.
- At the end, you should see a message like *"Kernel: arch/x86/boot/bzImage is ready"*.
- If errors occur, you'll need to resolve them (often due to missing dev packages or incompatible configuration options).

Linux Kernel

Kernel Compilation and Installation (Classic Method)

Step 2: Installing Kernel Modules

- Once the kernel is compiled, you need to install the kernel modules (those configured with "[M]").
- This step copies all generated `.ko` files (kernel objects) into the appropriate system directory (`/lib/modules/<kernel-version>/`).
- Run the command with root privileges:

```
sudo make modules_install
```

- This will install each module in the correct path and update the module index (via `depmod`) for the target kernel.
- You'll see a list of installed modules and a **DEPMOD** message at the end.
- Afterwards, the directory `/lib/modules/$(make kernelrelease)` contains all modules for the new kernel.
- **Note:** if you're compiling a kernel with the **same version** as an already installed one, `make modules_install` may overwrite the existing modules directory.
- To avoid this, ensure your *EXTRAVERSION* or *LOCALVERSION* is different, or move/rename the old modules directory (e.g., save it as `.old`).

Linux Kernel

Kernel Compilation and Installation (Classic Method)

Step 3: Installing the Kernel Image

- Next, install the kernel itself and its related files into `/boot`.
- If your system uses an initramfs, you must also generate this initial RAM disk.
- The easiest way is to use the `make install` target, which copies the kernel and triggers the necessary hooks:

```
sudo make install
```

- This copies the kernel image (`bzImage`) to `/boot/vmlinuz-<version>`, and installs the corresponding **System.map** (symbol table) and often the config file (`/boot/config-<version>`).
- On modern systems (e.g., Debian/Ubuntu), `make install` automatically triggers initramfs generation and bootloader updates via post-install scripts.
- You'll see scripts like `/etc/kernel/postinst.d/initramfs-tools` (which runs `update-initramfs`) and `.../zz-update-grub` (which runs `update-grub`).
- The **initrd** (or **initramfs**) is an initial RAM disk containing drivers needed to mount the root filesystem (e.g., disk controller drivers, FS drivers, etc.).
- It’s usually found under `/boot/initrd.img-<version>`.
- The **System.map** file is the kernel’s symbol table, useful for debugging and tools like `klogd`.

Linux Kernel

Kernel Compilation and Installation (Classic Method)

- After these steps, your new kernel is installed.
- You can verify everything is in place under `/boot`: you should see `vmlinuz-<version>` (the kernel binary), `initrd.img-<version>` (if required), and `System.map-<version>`.

Example:

```
ls -l /boot
# ... vmlinuz-5.15.8, initrd.img-5.15.8, System.map-5.15.8, config-5.15.8 ...
```

- Now, simply **reboot** into the new kernel.
- In the GRUB menu, select the new entry (usually chosen by default if it's the latest).
- After booting, confirm the running kernel version with `uname -r`, which should show the version you just compiled.
- For example, after successfully installing kernel 4.11.8, `uname -a` will show the new version and compilation date.

Congratulations, you have compiled and booted into your own custom Linux kernel!

Linux Kernel

Kernel Compilation and Installation (Classic Method)

Module and Firmware Management:

- Kernel modules are installed in `/lib/modules/<version>/`.
- You can load a module (optional driver) anytime using `modprobe <module_name>`, or unload it with `rmmod`.
- Don’t forget to add important modules that need to be loaded at boot (e.g., via `/etc/modules` on Debian) if you compiled essential components as modules instead of built-in.
- Regarding **firmware** (microcode required by certain devices like Wi-Fi cards, GPUs, etc.), note that these are usually **not included** in the kernel source for licensing reasons.
- Many drivers expect to load a firmware file from `/lib/firmware` when initializing hardware.
- Make sure to install the appropriate firmware packages for your distribution if needed (e.g., `firmware-linux-nonfree` on Debian for common proprietary firmware).
- It’s also possible to embed specific firmware directly into the kernel image by enabling **CONFIG_FIRMWARE_IN_KERNEL** and specifying which files to include, but the easiest method remains placing the required firmware files into `/lib/firmware` (or using official packages).
- If a firmware is missing, the driver will report it in the logs (`dmesg`) when the module loads — you’ll see an error like: “`firmware: failed to load ...`”.

Linux Kernel

Integration of Specific Drivers and Tools

- In some cases, you may need to add additional components to the kernel:
  - For example, an **external hardware driver** (not included in the official sources) or a **third-party module** (like proprietary NVIDIA drivers, VirtualBox modules, etc.).

Linux Kernel

Integration of Specific Drivers and Tools

Compiling External Modules with DKMS:

- **DKMS (Dynamic Kernel Module Support)** greatly simplifies the management of out-of-tree modules.
- It automatically compiles a third-party module for every installed kernel version and recompiles it when the kernel is updated.
- On Debian, many external drivers are available as packages ending in `-dkms`.
- For example, to add additional iptables modules, installing `xtables-addons-dkms` will compile and install the module automatically for the current kernel (provided the matching kernel headers are installed).
- DKMS fetches the module source code (often located under `/usr/src/<module>-<version>/`) and integrates it into the running kernel.
- You can check the status with `dkms status`.
- **Advantage:** every time a new kernel is installed, DKMS will recompile the registered external modules automatically. This is very helpful for proprietary drivers like NVIDIA, whose installer can register the module with DKMS to ensure compatibility across kernel updates.

Linux Kernel

Integration of Specific Drivers and Tools

Adding Drivers Not Included by Default:

- If the driver you want isn't available via a DKMS package, you have two main options:

(1) Simple Out-of-Tree Compilation:

- If you have the module's source code (e.g., a `driver.c` file with a Makefile from the vendor), you can compile it using the kernel headers.
- Install the appropriate `linux-headers-<version>` package (if you compiled the kernel yourself, you can install headers with `make headers_install`).
- From the module's directory, compile it with:

```
make -C /usr/src/linux-headers-<version> M=$(pwd) modules
```

- This produces a `.ko` file you can load using `insmod`, or install into `/lib/modules/<version>/extra` and load with `modprobe`.

Linux Kernel

Integration of Specific Drivers and Tools

(2) Recompiling a Patched Kernel:

- Use this method if the driver requires deep kernel integration or source code patching. Many projects distribute patch files for the vanilla kernel source.
- For example, Debian offers `linux-patch-*` packages (for security, real-time support like GrSecurity, etc.).
- To apply a patch, place the `.patch` or `.diff` file in the source directory and run:

```
patch -p1 < my_patch.diff
```

- Debian patches are often compressed (e.g., `.gz` in `/usr/src/kernel-patches/`). You can apply them with `zcat`, like this:

```
cd ~/kernel/linux-5.10.8
make clean
zcat /usr/src/kernel-patches/diffs/mypatch/patchfile.gz | patch -p1
```

Linux Kernel

Integration of Specific Drivers and Tools

Testing and Validating New Modules:

- After compiling an external module (with or without DKMS), you should test it.
- Use `modinfo <module.ko>` or `modprobe -n <name>` to check the module's metadata or simulate a load.
- Load it with `sudo modprobe <module_name>` and confirm it's active using `lsmod`.
- Watch `dmesg` or `/var/log/kern.log` for kernel messages — the driver may log info or errors (e.g., missing symbols or firmware).
- If you're replacing an existing module (e.g., testing a newer version of a network driver), you may need to blacklist the old one or pass a specific option to `modprobe`.
- If successful, verify the module's effect: e.g., a network interface appears, a file system mounts, etc.
- To make it persist across reboots, install the `.ko` in `/lib/modules/<version>/...`, run `depmod -a`, and ensure it's included in the initramfs or boot-time config if required.

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Loadable Kernel Modules (LKM)

Loadable Kernel Modules (LKM)

Designing a Kernel Module

A **Loadable Kernel Module (LKM)** is a dynamic extension to the Linux kernel.
It allows features to be added or removed on the fly, without the need to recompile or reboot the kernel.

Loadable Kernel Modules (LKM)

Designing a Kernel Module

- A **Linux kernel module** is a piece of code that can be dynamically loaded or unloaded into the running kernel.
- This allows features (often device drivers) to be added or removed without recompiling or restarting the kernel.
- Unlike components built *into* the kernel, loadable modules (LKMs) can be inserted on demand and removed when no longer needed, making the kernel more modular and lightweight.

Loadable Kernel Modules (LKM)

Designing a Kernel Module

Principle of Loadable Kernel Modules

Advantages:

- Modularity: only load the modules you need.
- Flexibility: add or remove features at runtime.
- Simplified maintenance: no need to recompile the entire kernel to add a driver.

Loadable Kernel Modules (LKM)

Designing a Kernel Module

- **A kernel module must define at least two special functions:**
  - An initialization function called when the module is loaded, and a cleanup function called just before it is unloaded.
- Historically, these were named `init_module` (called by `insmod`) and `cleanup_module` (called by `rmmod`).
- Since kernel 2.4, it's more common to use the macros `module_init()` and `module_exit()` to associate custom-named functions with module loading and unloading.
- The init function typically registers resources or handlers within the kernel (e.g., registering a device driver), while the cleanup function releases those resources so the module can be cleanly removed.

Loadable Kernel Modules (LKM)

Designing a Kernel Module

- When implementing a module, it must include the necessary kernel headers, especially `<linux/module.h>` (required for all modules) and often `<linux/kernel.h>` (for logging macros like `KERN_INFO`).
- The kernel's logging function `printk()` is used to output messages (standard I/O functions like `printf` are not available in kernel space).
- It's also recommended to define metadata macros (license, author, description, etc.) that can be viewed with `modinfo`.

Loadable Kernel Modules (LKM)

Designing a Kernel Module

Here is a minimal example of a simple module that prints a message on load and unload:

```
#include <linux/module.h>
#include <linux/kernel.h>

static int __init hello_init(void) {
    printk(KERN_INFO "Hello, kernel module!\n");
    return 0;  // 0 = successful load
}

static void __exit hello_exit(void) {
    printk(KERN_INFO "Goodbye, kernel module!\n");
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Simple example module printing messages");
```

Loadable Kernel Modules (LKM)

Kernel Module Management Commands

Here are the basic commands for managing kernel modules:

Command	Function
`lsmod`	List currently loaded modules
`modprobe`	Load/unload modules with automatic dependency handling
`insmod`	Load a specific module (no dependency resolution)
`rmmod`	Remove a module (without handling dependencies)
`modinfo`	Display detailed information about a module

Loadable Kernel Modules (LKM)

Compiling and Installing a Module

To compile an external module on Debian, you need the appropriate kernel headers for your current kernel, and build tools.

- Start by installing the required packages:
  - **Kernel headers and build tools**:
    Example for the current kernel:

    ```
    sudo apt update && sudo apt install build-essential linux-headers-$(uname -r)
    ```

  - `build-essential` provides `make`, `gcc`, etc., and `linux-headers-$(uname -r)` installs the development headers for your running kernel.
- **Module source code**:
  - Place the module source (e.g. `hello.c`) in a dedicated folder.
  - Write a **Makefile** to automate the build process.
  - A minimal Makefile uses the kernel's build system, specifying the headers directory and the module object.

Loadable Kernel Modules (LKM)

Compiling and Installing a Module

Example Makefile:

```
# Makefile to compile hello.ko
obj-m := hello.o
KDIR := /lib/modules/$(shell uname -r)/build
PWD  := $(shell pwd)

all:
	$(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
	$(MAKE) -C $(KDIR) M=$(PWD) clean
```

- `obj-m := hello.o` tells the kernel to build a module from `hello.o`.
- `$(MAKE) -C $(KDIR) M=$(PWD) modules` invokes the kernel's build system.
- This should generate a `hello.ko` file (the compiled kernel module).

Loadable Kernel Modules (LKM)

Compiling and Installing a Module

To compile the module:

- Run `make` in the directory with the code and Makefile.
- The kernel build system compiles the module using the running kernel's config.
- If successful, a `.ko` file will be generated.
- You can inspect the module with `modinfo` before loading it:

```
$ modinfo hello.ko
filename:       /home/user/hello/hello.ko
license:        GPL
description:    "Simple example module"
depends:
```

Loadable Kernel Modules (LKM)

Installing the Module (Optional)

- To make the module available for `modprobe` and auto-loading, copy it into an appropriate directory under `/lib/modules/$(uname -r)/` (e.g. `/extra/`).
- Then update the module index with:

```
sudo depmod
```

- For temporary use, you can load the module directly from the current folder without installation.

Loadable Kernel Modules (LKM)

Loading / Unloading a Module

Load a module:

```
sudo insmod hello_module.ko
```

Check if it's loaded:

```
lsmod | grep hello_module
```

Check kernel logs:

```
dmesg | tail
```

Unload the module:

```
sudo rmmod hello_module
```

View logs after unloading:

```
dmesg | tail
```

Loadable Kernel Modules (LKM)

Listing All Available Modules

All modules available for the current kernel are located in:

```
/lib/modules/$(uname -r)/
```

To list all existing modules:

```
find /lib/modules/$(uname -r)/ -name '*.ko*'
```

Loadable Kernel Modules (LKM)

Currently Loaded Modules

To see which modules are currently loaded in the kernel, use:

```
lsmod
```

- `lsmod` simply reads `/proc/modules` and displays it in a human-readable format.
- Each line has:
  - **Module** – module name.
  - **Size** – memory size in bytes.
  - **Used by** – usage count and dependent modules.

Loadable Kernel Modules (LKM)

Example Output of `lsmod`

```
$ lsmod | head -n5
Module                  Size  Used by
lp                     20480  0
ppdev                  24576  0
parport                53248  2 lp,ppdev
usbhid                 57344  0
```

- *lp* and *ppdev* are loaded but unused (0).
- *parport* is used by *lp* and *ppdev*.
- *usbhid* is loaded but not used by others.

Loadable Kernel Modules (LKM)

Notes on `lsmod` and `/proc/modules`

- `lsmod` is useful for quickly checking which drivers/features are active.
- Equivalent to reading `/proc/modules`.
- Only **dynamic modules** are listed — built-in kernel features are not shown.

Loadable Kernel Modules (LKM)

Viewing Module Info with `modinfo`

To get details about a kernel module (whether loaded or just available), use:

```
modinfo <module_name>  # or
modinfo /path/to/module.ko
```

Example with a common built-in module:

```
$ modinfo usbcore
filename:       /lib/modules/5.10.0-19-amd64/kernel/drivers/usb/core/usbcore.ko
description:    USB core driver
author:         {See file}
license:        GPL
alias:          usb-host-class-device
srcversion:     5C6FF1234567890ABCDEF
depends:        usb-common
intree:         Y
name:           usbcore
vermagic:       5.10.0-19-amd64 SMP mod_unload modversions
```

Loadable Kernel Modules (LKM)

Displaying module information

You get a lot of useful information:

- The exact path to the module file (`filename`), a textual description, the author, the license, any aliases (alternative names used by the system, for example for auto-loading via udev), the list of *dependencies* (`depends`) meaning other modules that must be loaded for it to function, the "vermagic" (version magic) which must match the kernel version, etc.
- Most of these fields correspond to macros inserted by the module developer (`MODULE_DESCRIPTION`, `MODULE_AUTHOR`, `MODULE_LICENSE`, `MODULE_ALIAS`, etc.).
- You can use `modinfo` on a module that is *already loaded* or one that is simply installed on the disk.
- If the module is loaded, the info comes from the file on disk (it does not query the kernel directly).
- For instance, `modinfo hello.ko` (our example module) or `modinfo hello` (if installed in `/lib/modules`) will show the GPL license and the description we defined in the code, confirming that the module matches our expectations.

Loadable Kernel Modules (LKM)

Module dependency management

- Kernel modules can depend on each other.
- For example, a wireless network device module might depend on a crypto stack module or a PCI bus module.
- To automatically manage these **dependencies**, Linux uses an index file called `modules.dep` that lists, for each module, the set of its possible dependencies.

Loadable Kernel Modules (LKM)

Module dependency management

Generating modules.dep (depmod):

- The file `/lib/modules/<version>/modules.dep` is generated by the `depmod` utility.
- Each time you install or update modules (e.g., after installing a new kernel or a third-party module), you run `depmod -a` to scan all available modules and recalculate their dependencies.
- `depmod` reads each `.ko` file and determines which symbols or other modules it needs, then writes the result into `modules.dep` (and a binary file `modules.dep.bin`).
- This allows other tools to instantly know dependencies without analyzing the files every time.
  For example, if module *abc.ko* needs modules *def.ko* and *ghi.ko*, `modules.dep` will contain a line like:

```
/lib/modules/5.10.0-19-amd64/.../abc.ko: /lib/modules/5.10.0-19-amd64/.../def.ko /lib/modules/5.10.0-19-amd64/.../ghi.ko
```

  telling `modprobe` to load *def* and *ghi* before *abc*.

Loadable Kernel Modules (LKM)

Module dependency management

Using `modprobe` vs `insmod`:

- Thanks to `modules.dep`, the `modprobe` command can automatically load all the modules required by a target module.
- For instance, if *foo* depends on *bar* and *baz*, a simple `modprobe foo` will insert *bar*, *baz*, then *foo* in the correct order.
- That's why it is **recommended to use `modprobe`** instead of `insmod` for loading modules.
- `insmod` does no resolution at all:
  - If you insert *foo.ko* without having previously inserted its dependencies, the call will fail (or the module will crash at runtime).
  - In practice, `insmod` is used for quick tests when you know exactly the loading order, or for very specific cases.
  - For regular use, `modprobe` handles everything by relying on `modules.dep`.

Loadable Kernel Modules (LKM)

Module dependency management

Updating dependencies:

- On Debian, when installing a new kernel or module, the install script usually runs `depmod -a` automatically.
- However, if you manually add a `.ko` file in `/lib/modules/...`, don't forget to run `sudo depmod -a` before using `modprobe` or `modinfo` on that module.
- Otherwise, `modinfo` or `modprobe` may fail to find it or resolve its dependencies, as `modules.dep` won't be up to date (leading to *"Module not found"* errors).

Loadable Kernel Modules (LKM)

Blocking a module

- Sometimes you may want to prevent a particular module from loading, for example to disable a problematic driver or one that conflicts with another.
- There are two main ways to **blacklist** a module on Debian: via the modprobe configuration or using kernel boot options.

Loadable Kernel Modules (LKM)

Blocking a module

Blacklisting via modprobe config file:

- You can prevent a module from loading automatically by listing it in a modprobe configuration file.
- The common file is `/etc/modprobe.d/blacklist.conf` (or you can create a separate `.conf` file).
- Simply add a line like:

```
blacklist <module_name>
```

- For example, to blacklist the `nouveau` module (open-source Nvidia driver), add `blacklist nouveau`. Then, it's recommended to run `depmod -ae` and regenerate the initramfs: `sudo update-initramfs -u`.
- This ensures the module won't be loaded automatically at next boot, even during the initramfs stage.
- **Warning**: modprobe blacklist prevents *automatic* loading of the module (by udev or others), but does **not** stop an admin from manually loading it.
- In fact, `sudo modprobe <module>` will force load even if blacklisted.
- To fully prevent insertion, even manually, you can use an *install* directive to redirect to `/bin/false`. For example, in `/etc/modprobe.d/blacklist.conf`:

```
install <module_name> /bin/false
```

Loadable Kernel Modules (LKM)

Blocking a module

Blacklisting via kernel boot parameter (GRUB):

- The other method is to pass a boot parameter to the Linux kernel so that it ignores a module. Use the syntax `modprobe.blacklist=<module_name>` in the kernel boot line.
- On Debian, you can edit the file `/etc/default/grub` and add the module(s) to blacklist in the `GRUB_CMDLINE_LINUX` variable.
- For example:

```
GRUB_CMDLINE_LINUX="modprobe.blacklist=nouveau"
```

  - You can add it to existing parameters, separated by spaces.
  - For multiple modules, list them comma-separated, e.g., `modprobe.blacklist=nouveau,firewire_ohci`.
  - Then run `sudo update-grub`.
  - On next reboot, the kernel will not load these modules at startup.
  - This is equivalent to the previous method but takes effect earlier in the boot process, before initramfs loads anything.
  - This technique is often used to disable *nouveau* before installing the proprietary Nvidia driver.

Loadable Kernel Modules (LKM)

Building a Custom Kernel

- In certain situations, you may want to compile a custom kernel where some modules are directly **built into** the kernel image (instead of being separate modules).
- On Debian, it is entirely possible to compile your own kernel while still keeping a clean `.deb` package format for installation.

Loadable Kernel Modules (LKM)

Building a Custom Kernel

1) Prepare the build environment – Install the required tools:

```
sudo apt install build-essential kernel-package fakeroot libncurses-dev
```

- *build-essential* (gcc, make, etc.) and *libncurses-dev* (for the kernel's text-based config UI) are essential.
- *kernel-package* provides the `make-kpkg` tool for building Debian-style kernels.
- *fakeroot* allows package building without needing root privileges.

Loadable Kernel Modules (LKM)

Building a Custom Kernel

2) Get the kernel sources – Two approaches:

- Either retrieve the sources provided by Debian (`linux-source-<version>` package),
- Or download the desired version from kernel.org.
- For a Debian kernel, it's recommended to use `apt`: e.g., `sudo apt install linux-source-5.10`.
- This will place an archive (typically in `/usr/src/`) that you'll need to extract:

```
cd /usr/src/
tar xf linux-source-5.10.tar.xz
cd linux-source-5.10
```

Loadable Kernel Modules (LKM)

Building a Custom Kernel

3) Configure the kernel (make menuconfig) –

- Before compiling, you need to select which components will be included.
- You can start from the current Debian kernel config (`/boot/config-$(uname -r)`) by copying it to `.config` and then running `make oldconfig` or `make menuconfig`.
- The `make menuconfig` command opens a text-based interface that lets you browse and adjust all kernel options.
- This is where you choose whether to compile a feature as a built-in or as a module.
- Navigate to the desired option and change its state:
- `[*]` (Y) = built directly into the kernel (builtin)
- `[M]` = built as a loadable module
- `[ ]` (N) = not included at all

Loadable Kernel Modules (LKM)

Building a Custom Kernel

4) Compile the kernel and modules –

- Now use `make-kpkg` to compile and package the kernel.
- This tool automates compilation (`make`) and creates a .deb package.
- For example:

```
export CONCURRENCY_LEVEL=4    # optional: speeds up build on multi-core CPUs
fakeroot make-kpkg --initrd --revision=1.0-custom kernel_image kernel_headers
```

- This command builds the kernel with an initrd image (`--initrd`) and assigns a custom revision number.
- It will produce two `.deb` files in the parent directory: one for the kernel image (`linux-image-...custom_amd64.deb`) and one for the headers (`linux-headers-...custom_amd64.deb`).
- You can tweak `--revision` to tag your custom build.
- Compilation may take a while depending on your system's power and the selected features/modules.

Loadable Kernel Modules (LKM)

Building a Custom Kernel

5) Install the custom kernel –

- Once the `.deb` packages are built, install them using `dpkg`:

```
sudo dpkg -i ../linux-image-5.10.0-custom_amd64.deb ../linux-headers-5.10.0-custom_amd64.deb
```

- The `linux-image` package will place the kernel file (`vmlinuz-5.10.0-custom`) in `/boot` and the compiled modules in `/lib/modules/5.10.0-custom/`.
- It will also generate an initrd with required modules (thanks to `--initrd`) and automatically update **GRUB** to add a new entry for this kernel (via Debian's post-install scripts).
- The `linux-headers` package isn't strictly necessary to boot the kernel, but it's helpful if you want to build external modules later for this version.

Loadable Kernel Modules (LKM)

Booting into the Custom Kernel

6) Reboot into the new kernel

- Update GRUB if needed (`sudo update-grub`) – this is usually done automatically – then reboot and select your custom kernel from the boot menu.
- Once the system has started, you can verify with `uname -r` that you're running the custom version.
- Your **built-in modules** will appear as part of the kernel: they **won't show up in `lsmod`** (since they are not dynamically loaded).
- For instance, if you had previously used a driver as a module and now compiled it into the kernel, `lsmod` won't list it anymore – but the functionality will still be present (and can often be seen via `/proc/devices` or other tools).

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

"/proc" and "/sys"

"/proc" and "/sys"

Overview of the /proc Pseudo-filesystem

- The `/proc` directory is a **pseudo-filesystem** that exposes kernel-related information.
- It's referred to as a *pseudo*-filesystem because its contents are generated **dynamically in memory** by the kernel, rather than being permanently stored on disk.
- In practice, reading a file within `/proc` directly queries the kernel for the requested data.
- These files appear empty (`0` bytes) when listed with `ls`, which highlights that they don't occupy any real disk space (only a small amount of RAM is used).
- The `/proc` pseudo-filesystem serves as an **interface between the user and the kernel's internal data structures**.
- It was originally designed to provide information about currently running processes — hence the name *proc* (short for "process") — but today it encompasses a wide range of system-level data.
- On Debian (and most Linux distributions), `/proc` is **automatically mounted** at boot time by the system (typically by the kernel or the init process).
- It is normally always available without manual intervention.
- As a reference, you could manually mount it using `mount -t proc proc /proc`, but this is rarely necessary since the system handles it automatically at startup.

"/proc" and "/sys"

Contents of `/proc`

- The contents of `/proc` are structured as virtual files and directories reflecting the system's current state.
- It contains both **global files** describing the overall system and **per-process directories**.
- Most of these files are read-only (for inspection), but some can be modified to adjust kernel parameters (see the sysctl section).

"/proc" and "/sys"

Contents of `/proc`

- **`/proc/cpuinfo`** – Details about the CPU: model, frequency, number of cores, features, etc. You can view it using `cat /proc/cpuinfo`.
- **`/proc/meminfo`** – Memory and swap info: total, free, cached, buffer, etc. Tools like `free` rely on this.
- **`/proc/uptime`** – System uptime. The two numbers represent time since boot and total idle time.
- **`/proc/version`** – Kernel version and build details.
- **`/proc/filesystems`** – Lists supported filesystems.
- **`/proc/interrupts`** – Hardware interrupt counters by CPU (useful to see IRQ usage).
- **`/proc/swaps`** – Active swap areas and usage.
- **`/proc/loadavg`** – System load averages (1, 5, 15 min), running processes, latest PID.

"/proc" and "/sys"

Contents of `/proc`

- To **view these files**, use standard shell commands.
- Example: `cat /proc/cpuinfo` shows processor info.
- You can combine with `grep` to filter data:

```
$ grep "model name" /proc/cpuinfo
```

- This returns the model line for each CPU.
- Similarly, `grep MemTotal /proc/meminfo` gives the total RAM.
- Use `find` to explore `/proc`:

```
find /proc -maxdepth 1 -name "*meminfo*"
```

- While you often know the exact file path, `find` helps with less common entries.

"/proc" and "/sys"

Per-process Files (`/proc/<PID>/` and `/proc/self/`)

- Each active process has a corresponding directory under `/proc`, named after its PID.
- For instance, process 1234 has `/proc/1234`.
- These directories provide info specific to the process:
  - **`/proc/<PID>/cmdline`** – Command line used to start the process.
  - **`/proc/<PID>/status`** – Readable summary: user, memory usage, parent PID, state, etc.
  - **`/proc/<PID>/cwd`** – Symlink to current working directory (`pwdx <PID>` uses this).
  - **`/proc/<PID>/exe`** – Symlink to the running executable.
  - **`/proc/<PID>/fd/`** – Directory of open file descriptors (each entry links to an open file/resource).
  - **`/proc/<PID>/maps`**, **`smaps`** – Details about memory mappings.
  - **`/proc/<PID>/task/`** – Lists threads of the process (each with its own TID directory).

"/proc" and "/sys"

File Access Rights

- Most of `/proc` is world-readable, but some files have access restrictions.
- For instance, a user typically cannot inspect another user's process details (especially with `hidepid` mount options enabled).
- Likewise, modifying files (like those under `/proc/sys`) requires superuser privileges.

"/proc" and "/sys"

Modifying Kernel Parameters with `sysctl`

The `sysctl` utility lets you read and change kernel parameters on the fly.

- These parameters control system behavior in areas such as:
  - **Networking** (e.g., TCP/IP, routing, firewall)
  - **Memory management**
  - **System security**
  - General system performance

"/proc" and "/sys"

Modifying Kernel Parameters with `sysctl`

- The Linux kernel exposes modifiable parameters under `/proc/sys/`.
- These *sysctl variables* can be viewed and changed at runtime via the **`sysctl`** command.
- `sysctl` provides a cleaner CLI interface than manually editing files under `/proc/sys`.
- Behind the scenes, `sysctl` directly reads/writes to procfs entries.

"/proc" and "/sys"

Modifying Kernel Parameters with `sysctl`

To **list all available parameters**, use:

```
$ sysctl -a
```

- This outputs all sysctl keys and their current values.
- The list spans many subsystems:
  - Core kernel (`kernel.*`),
  - Networking (`net.*`),
  - Virtual memory (`vm.*`),
  - Security and filesystems (`fs.*`).
- Use filters like:

  ```
  sysctl -a | grep '^net.'
  ```

  or

  ```
  sysctl --pattern '^net.'
  ```

"/proc" and "/sys"

Modifying Kernel Parameters with `sysctl`

To **view a specific key**, use its full name:

```
$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
```

- This shows whether IP forwarding is enabled.
- Default is usually 0 (disabled).
- Equivalent manual check:

```
cat /proc/sys/net/ipv4/ip_forward
```

"/proc" and "/sys"

Modifying Kernel Parameters with `sysctl`

- **Temporarily change a value:**

```
sudo sysctl -w net.ipv4.ip_forward=1
```

  - Effective immediately, but lost after reboot.

- **Make it permanent:**

  Edit `/etc/sysctl.conf`:

```
sudo nano /etc/sysctl.conf
```

  Add:

```
net.ipv4.ip_forward = 1
```

- **Apply changes now:**

```
sudo sysctl -p
```

"/proc" and "/sys"

Overview of the `/sys` pseudo-filesystem

- The `/sys` directory corresponds to the pseudo-filesystem known as **sysfs**.
- Introduced with the Linux 2.6 kernel, its purpose is to expose a unified view of **hardware devices and their drivers** to user space.
- Like `/proc`, it is a virtual filesystem maintained in memory (originally based on *ramfs*).
- However, its content and purpose differ from `/proc`: `/sys` is organized according to the kernel's internal structure (the *Device Tree*) and is mainly intended to represent the system's hardware configuration.
- Historically, before Linux 2.6, the `/proc` directory had started to accumulate information not directly related to processes (such as hardware data, drivers, etc.), making it harder to read.
- Sysfs was designed to **relieve `/proc` by moving all device-related information** and kernel subsystems into `/sys`.
- In practice, `/sys` provides a hierarchical view of all hardware components (buses, devices, drivers...) in the system, separate from the process-related information found in `/proc`.

"/proc" and "/sys"

Overview of the `/sys` pseudo-filesystem

- Like `/proc`, the `/sys` pseudo-filesystem is mounted automatically at boot.
- On systems using **systemd** as init system, it is mounted very early during the boot sequence.
- On traditional Debian/Ubuntu systems (SysVinit or systemd), you'll often also find an entry in `/etc/fstab` like: `sysfs /sys sysfs defaults 0 0`, indicating that sysfs should be mounted on `/sys`.
- In practice, regardless of the distribution (Debian, Arch, Fedora...), **`/sys` is always mounted by the system at boot**, as critical components like udev rely on it to detect hardware.
- Therefore, it is not necessary to mount it manually (except in minimalist environments or in chroot, if applicable).

"/proc" and "/sys"

Information contained in `/sys`

The structure of `/sys` is hierarchical and reflects the internal architecture of the kernel. At the root of `/sys`, you'll notably find:

- **`/sys/devices/`** – Groups physical devices according to hardware hierarchy.
  - This shows the actual device tree: for example, under `/sys/devices/system/cpu/` you'll find CPUs, under `/sys/devices/pci0000:00/` the devices on the root PCI bus, etc.
  - This is where all detected system devices (disks, network interfaces, USB, etc.) appear, organized by bus and connections.
- **`/sys/class/`** – Logical view by *device class*.
  - Classes group devices with similar functions, regardless of their position on a specific bus.
  - For example, the `net` class contains all network interfaces on the system (eth0, wlan0, lo, etc.), the `block` class includes all block storage devices (sda, sdb, loop0, ...), and the `tty` class contains terminals, etc.
  - Browsing `/sys/class` lets you quickly find a device type without knowing its exact position under `/sys/devices`.

"/proc" and "/sys"

Information contained in `/sys`

- **`/sys/bus/`** – View by system bus.
  - This directory contains an entry for each bus type present in the kernel (pci, usb, spi, i2c, etc.).
  - Inside, you can see the devices attached to each bus (`/sys/bus/pci/devices/...` for example) as well as their associated drivers (`/sys/bus/pci/drivers/...`).
  - It offers another way to access the same information, but organized by communication bus.
- **`/sys/modules/`** – Information about loaded kernel modules.
  - Each module (driver or dynamically loaded kernel component) has a folder here, typically containing a `parameters/` file exposing the module's configurable parameters.
  - For example, if a driver module accepts options (like buffer size, etc.), they will be visible and editable via files in `/sys/modules/<module_name>/parameters/`.
- **`/sys/kernel/`** – Data specific to the kernel itself.
  - This includes debug info (`/sys/kernel/debug/` if mounted), security settings (`/sys/kernel/security/`), and runtime kernel configuration (e.g., `/sys/kernel/mm/` for memory).
  - This directory can also host other pseudo-filesystems such as *configfs* (usually mounted on `/sys/kernel/config`).

"/proc" and "/sys"

The `sysTool` (systool) utility

- `systool` (not to be confused with *sysctl*) is a command-line tool used to query the sysfs hierarchy more conveniently.
- Provided by the `sysfsutils` package on Debian/Ubuntu, it is used to list devices and their attributes by bus, class, or kernel module, using the libsysfs API.
- In other words, `systool` offers a formatted view of what is available under `/sys`, which can be more convenient than browsing directories manually.

"/proc" and "/sys"

`sysTool` (systool) utility

- **Overview and installation:**
  - On Debian, to use `systool`, you need to install the **`sysfsutils`** package (if it's not already installed).
  - This package also includes a configuration file `/etc/sysfs.conf` that allows setting values to be written to certain sysfs nodes at system startup.
  - On Arch Linux and Fedora, `systool` is also not installed by default, but it is available in the `sysfsutils` package (installable via `pacman`), and in the official repositories for Fedora (installable via `dnf`/`yum`).

"/proc" and "/sys"

`sysTool` (systool) utility

Common usage:

- Without any arguments, the `systool` command displays all bus types, all device classes, and all root devices available on the system. This provides an overview of the hardware hierarchy.
- You can refine the query with options:
- `systool -b <bus>` – Lists devices and information for a given bus. For example, `systool -b pci` displays the list of PCI devices and their attributes (identifiers, resources, associated driver, etc.).
- `systool -c <class>` – Displays devices of a specific **class**.
- `systool -m <module>` – Provides information about a loaded **kernel module**. This is very useful for viewing driver parameters.

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Hardware Troubleshooting

Hardware Troubleshooting

Types of Hardware Issues

Servers may encounter various common hardware problems affecting the **CPU**, **RAM**, **storage**, **power supply**, or **network**.

- For example, an overheating or failing CPU can cause slowdowns, **kernel panics**, or sudden system shutdowns.
- A **RAM** failure (e.g., a faulty stick) may lead to random errors such as processes terminating unexpectedly, **segmentation faults**, or kernel crashes that appear software-related but are in fact due to hardware issues.
- A failing **storage device** (HDD/SSD) may generate **I/O** (input/output) errors in the logs, unreadable sectors, or severely degraded performance. A defective power supply can cause spontaneous reboots or prevent the server from booting at all.
- A faulty **network card** can lead to network drops, loss of connectivity, or corrupted packets.

Hardware Troubleshooting

Hardware Analysis

Hardware analysis on Linux relies on various tools and techniques:

a) Inventory and Hardware Information Tools

- **`lspci`**
  Displays connected PCI devices.
  Example:

  ```
  lspci -v
  ```

  This command provides detailed descriptions of controllers, graphics cards, network interfaces, etc.

- **`lsusb`**
  Displays connected USB devices.
  Example:

  ```
  lsusb -v
  ```

  Useful to verify whether USB devices are detected and which drivers are associated with them.

Hardware Troubleshooting

Hardware Analysis

- **`lshw`**
  Provides a full inventory of the hardware, including configuration and capabilities.
  Example:

```
sudo lshw -short
```

  This command gives an easy-to-read summary and can be used to generate a complete report.

- **`inxi`**
  A comprehensive system information tool (often installed manually on Debian/Ubuntu) that gives an overview of hardware and system status.
  Example:

```
inxi -F
```

Hardware Troubleshooting

Hardware Analysis

- **`dmidecode`**
  Reads BIOS and DMI (Desktop Management Interface) information.
  Example:

  ```
  sudo dmidecode
  ```

  Useful to obtain details about the manufacturer, motherboard model, BIOS version, memory configuration, etc.

Hardware Troubleshooting

Hardware Analysis

b) Analyzing Hardware Issues in Logs

- `dmesg`
  Displays the kernel log, which contains many messages about hardware and drivers.
  Example:

```
dmesg | less
```

  Look for keywords like *error*, *fail*, *timeout*, or *warn* to identify incidents. For example:

```
dmesg | grep -i error
```

  filters out error messages reported by the kernel.

Hardware Troubleshooting

Hardware Analysis

b) Analyzing Hardware Issues in Logs

- **`journalctl`** (for systems using systemd)
  Allows you to view system and kernel logs.
  Example:

```
sudo journalctl -k | grep -i usb
```

  This helps identify issues related to USB devices or other hardware components.

- **Specific log files**
  Some hardware problems may also be logged in `/var/log/syslog`, `/var/log/messages`, or `/var/log/kern.log` (depending on the distribution). These files can contain clues about hardware malfunctions.

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Logicial Volume Manager (LVM)

Logical Volume Manager (LVM)

Overview of Main Filesystems

- **ext2**: Older system without journaling, unreliable in case of crashes, rarely used today.
- **ext3**: Adds journaling to ext2, enabling faster recovery after a crash.
- **ext4**: Modern, fast, reliable filesystem supporting large file sizes (up to 1 EiB).
- **XFS**: Optimized for handling large files and massive storage systems, performs well on servers.
- **ZFS**: Advanced filesystem with data integrity, built-in RAID management, and integrated snapshots.

Logical Volume Manager (LVM)

Detailed Description of LVM and Device Mapper

- The *Logical Volume Manager (LVM)* is a logical volume management system available on Linux systems.
- Instead of using fixed partitions directly on a hard disk (or sets of disks), LVM allows the creation and management of more flexible logical volumes that are abstracted from the underlying hardware.
- This allows resizing, moving, or merging volumes more easily than with traditional partitions.

Logical Volume Manager (LVM)

Detailed Description of LVM and Device Mapper

- The *Device Mapper* is a Linux kernel component (an abstraction layer) that provides the foundation for creating and managing various virtual storage devices.
- It offers a generic mapping mechanism between virtual block devices and real physical block devices.
- LVM tools rely on the Device Mapper to create their logical volumes.

Logical Volume Manager (LVM)

Detailed Description of LVM and Device Mapper

- **The Device Mapper** is the core kernel component that enables:
  1. Defining mapping tables to distribute real blocks (sectors, clusters) across one or more virtual devices (e.g., a logical volume).
  2. Implementing features like *snapshots*, block-level encryption (via dm-crypt), mirroring, etc.

Logical Volume Manager (LVM)

Managing Volume Groups (VG), Physical Volumes (PV), and Logical Volumes (LV)

LVM introduces an abstraction layer based on three main concepts:

1. **Physical Volumes (PV)**
   - These are physical disks or partitions (e.g., `/dev/sda1`, `/dev/sdb`, etc.) marked to be managed by LVM.
   - Each PV is divided into “Physical Extents” (PE), fixed-size storage units (default: 4 MB).

Logical Volume Manager (LVM)

Managing Volume Groups (VG), Physical Volumes (PV), and Logical Volumes (LV)

LVM introduces an abstraction layer based on three main concepts:

2. **Volume Groups (VG)**
   - A Volume Group is a collection of one or more PVs.
   - All Physical Extents (PEs) from those PVs form a common storage “pool”.
   - Once a VG is created, you can create as many logical volumes as the available space allows.

Logical Volume Manager (LVM)

Managing Volume Groups (VG), Physical Volumes (PV), and Logical Volumes (LV)

LVM introduces an abstraction layer based on three main concepts:

3. **Logical Volumes (LV)**
    - These are the actual logical volumes. They can be considered the “virtual” equivalent of a partition.
    - LVs are themselves divided into “Logical Extents” (LE), which, in practice, correspond to the underlying Physical Extents.
    - A filesystem (ext4, xfs, btrfs, etc.) or another type of service (such as swap) is installed on an LV.
    - LVs benefit from the flexibility to be resized, moved, cloned, and more.

Logical Volume Manager (LVM)

Managing Volume Groups (VG), Physical Volumes (PV), and Logical Volumes (LV)

LVM allows you to:

- Hot-add a new disk to a Volume Group, thereby increasing the total capacity of the storage "pool".
- Resize a logical volume (expand or shrink it, depending on the filesystem's constraints) without needing to physically restructure all disk partitions.
- Move extents from one disk to another — for example, to remove a physical disk from a group or to balance storage load.
- Create *snapshots* of volumes (*read-only* or *read-write* instant copies), which are particularly useful for backups or cloning purposes.

Logical Volume Manager (LVM)

Managing Volume Groups (VG), Physical Volumes (PV), and Logical Volumes (LV)

How LVM uses the Device Mapper

- At the kernel level, each Logical Volume managed by LVM is exposed as a virtual block device under `/dev/mapper/...` or sometimes `/dev/VGName/LVName`.
- Behind the scenes, LVM provides the *Device Mapper* with a mapping table that states: “this segment of the logical volume corresponds to this region on `/dev/sdb1`, another segment maps to `/dev/sdc1`, etc.”
- The Device Mapper is then responsible for aggregating these physical blocks into a single, coherent virtual block device.

Logical Volume Manager (LVM)

Physical Extents (PE) and Logical Extents (LE)

1. Physical Extents (PE)

1. **Definition:**
   Physical Extents are the smallest allocatable storage units on a *Physical Volume (PV)*.
2. **Fixed size:**
   - When creating a Volume Group (VG), you define the extent size (commonly 4 MB by default).
   - All PVs included in this Volume Group will be divided into PEs of the same size.
3. **Role:**
   - PEs are the foundation from which *Logical Volumes (LV)* draw their space.
   - By managing storage in blocks of a few megabytes (instead of at the sector or disk block level), LVM simplifies volume resizing, shrinking, and data relocation.

Logical Volume Manager (LVM)

Physical Extents (PE) and Logical Extents (LE)

2. Logical Extents (LE)

1. **Definition:**
   Logical Extents are the logical blocks that make up a *Logical Volume (LV)*.
2. **1:1 Mapping:**
   - The size of a Logical Extent is the same as that of a Physical Extent (defined at the Volume Group level).
   - Each LE maps directly to a specific PE.
   - Therefore: 1 LE ↔️ 1 PE.
3. **Role:**
   - LEs allow the logical volume to be represented in a continuous and uniform way, even if the underlying PEs are spread across different physical disks.
   - When a Logical Volume is “expanded” or “shrunk,” LVM adds or removes a number of LEs (which correspond to available or freed PEs on the PVs).

Logical Volume Manager (LVM)

Physical Extents (PE) and Logical Extents (LE)

3. How it works in practice

1. **Creating a Physical Volume (PV)**
   - For example, you initialize a disk or partition:

```
pvcreate /dev/sdb1
```

   - This allows LVM to treat it as a “reservoir” of PEs (once it is part of a Volume Group).
2. **Creating a Volume Group (VG)**
   - One or more PVs are aggregated into a VG:

```
vgcreate VG_DATA /dev/sdb1
```

   - During this step, you can specify (or LVM will choose by default) the extent size (e.g., `--extent-size 4M`).
   - The VG can then contain a large number of 4 MB PEs.

Logical Volume Manager (LVM)

Physical Extents (PE) and Logical Extents (LE)

3. How it works in practice

3. **Creating a Logical Volume (LV)**
   - You allocate a portion of the VG's space to create an LV:

     ```
     lvcreate -n LV_home -L 10G VG_DATA
     ```

   - LVM then associates a number of LEs equivalent to 10 GB (e.g., if 1 LE = 4 MB, you'll get 2560 LEs).
   - Each LE of this LV maps to a PE on one of the PVs in the VG.
4. **Scalability and flexibility**
   - To expand an LV, you simply add more LEs (which consumes additional PEs).
   - To move an LV, LVM can remap its LEs to other available PEs.

Logical Volume Manager (LVM)

Physical Extents (PE) and Logical Extents (LE)

4. Advantages of this approach

1. **Granular management:**
   By working with multi-megabyte blocks (instead of fixed partitions), LVM allows for easier storage reallocation without the need to repartition the disk.
2. **Flexibility:**
   You can extend an LV from the available space in the VG, even if that space spans multiple physical disks.
3. **Performance and maintenance:**
   Using extents simplifies data migration and maintenance operations (moving extents from one disk to another, adding/removing disks, etc.).

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

- In the context of LVM (Logical Volume Manager), each *Physical Volume* (PV) contains several metadata regions.
- Documentation and training materials often refer to them with names like **PVRA**, **VGRA**, or **BBRA**.
- These acronyms may vary across versions or vendors (some come from the HP-UX LVM legacy or other implementations), but the overall idea is the same:
  - they refer to reserved areas on the disk (or partition) that store critical information about the PV itself, the Volume Group,
  - and possibly other data (like a boot block or bad block management).

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

1. PVRA (Physical Volume Reserved Area)

1. **Main role**
   - The *Physical Volume Reserved Area* refers to the metadata region located at the beginning (or sometimes the end) of a disk or partition when it is initialized as a Physical Volume.
   - It typically contains:
     - The LVM *label* (a signature that marks the disk as a PV).
     - Basic information about the PV: its UUID, extent size, etc.
2. **Location and size**
   - This is usually just a few sectors or blocks reserved — typically at the start of the PV (when you run `pvcreate`, LVM writes its signature and basic data there).
   - This area is tiny compared to the size of the disk (a few kilobytes), but it is critical, as it allows the system to identify and access the PV correctly.

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

1. PVRA (Physical Volume Reserved Area)

3. **History and naming**
   - It is sometimes referred to as the “PV label” or “LVM header” in more recent documentation.
   - On legacy systems (like HP-UX LVM), the term “Physical Volume Reserved Area” was used to encompass all disk metadata related to PV status.

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

2. VGRA (Volume Group Reserved Area)

1. **Main role**
   - The *Volume Group Reserved Area* is the metadata region where configuration details of the *Volume Group* (VG) are stored.
   - Specifically, this is where LVM records the list of Logical Volumes, their sizes, their mapping to Physical Extents, etc.
   - It may also contain journaling information to preserve consistency in the event of a failure.
2. **Redundant copy**
   - When multiple disks are part of the same VG, LVM can store multiple copies of the VG metadata (one on each PV). This provides resilience against the loss of a single disk, as long as the metadata is duplicated.
   - The number of copies depends on the configuration (`metadata copies` parameter).

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

2. VGRA (Volume Group Reserved Area)

3. **Location flexibility**
   - Depending on the LVM version and configuration, this area can be located either at the beginning or end of the PV.
   - By default, LVM often places the metadata near the beginning, but this behavior can be changed using `--metadataarea` or `--metadataignore`.

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

3. BBRA (Boot Block or Bad Block Reserved Area)

- This acronym is less common in official LVM documentation but may appear in older or OS-specific resources (e.g., HP-UX, AIX) or training materials.
- Two main interpretations are commonly associated with it:

1. **Boot Block Reserved Area**
    - In certain implementations, this is a reserved area for storing boot information or a bootloader (in conjunction with LVM).
    - In rare cases where a system boots directly from an LVM volume (without a separate /boot), this space may be reserved for boot code or critical boot-time data.

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

3. BBRA (Boot Block or Bad Block Reserved Area)

2. **Bad Block Relocation Area**
    - On older systems or legacy disks, a dedicated area was sometimes reserved for *relocating* bad blocks.
    - LVM or the OS could mark these blocks as unusable and use this reserved area to remap them or store a bad block table.
    - Today, modern drives (SSDs or HDDs with onboard firmware) usually handle sector remapping internally, making this feature largely obsolete from LVM's perspective.

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

4. How this works in practice

- When you run `pvcreate /dev/sdb`:
  1. LVM writes the PV label + the PVRA at the beginning (or end) of `/dev/sdb`.
  2. It also reserves space to store the VGRA (even if you haven't created a Volume Group yet).
- When you run `vgcreate myVG /dev/sdb`:
  1. LVM initializes the "Volume Group metadata" (VGRA) in the reserved area.
  2. It stores information about the new VG (name, UUID, etc.).
- If the concept of **BBRA** is supported by your OS or distribution (now uncommon), it will also be initialized during `pvcreate` or via specific tools.

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

4. How this works in practice

In practice, most LVM users do not directly interact with these three zones:

- A **PV** is known to contain a label + metadata at the beginning (and sometimes at the end).
- The **VG metadata** (which describes LVs, PVs, etc.) is stored in a special area.
- The rest of the disk is divided into *Physical Extents* (PEs) used to store user data.

Logical Volume Manager (LVM)

Metadata (PVRA, VGRA, BBRA)

5. Summary

- **PVRA (Physical Volume Reserved Area)**
  Refers to the metadata block that marks a disk/partition as a Physical Volume and contains basic identification info (LVM signature, UUID, extent size, etc.).
- **VGRA (Volume Group Reserved Area)**
  Metadata area that describes the complete configuration of the Volume Group: list of LVs, attributes, mappings, journaling, etc.
  Can be replicated across multiple PVs in the same VG for redundancy.
- **BBRA (Boot Block / Bad Block Reserved Area)**
  A space that *could* be used for boot information or bad block relocation, mostly in older LVM implementations or specific UNIX systems. On modern Linux systems, this concept is rarely highlighted or used.

Logical Volume Manager (LVM)

Volume Security

- Securing LVM volumes relies on several complementary strategies that aim to protect both data integrity and confidentiality.

Logical Volume Manager (LVM)

Volume Security

1. Volume Encryption

- **Using LUKS and dm-crypt:**
  LUKS (Linux Unified Key Setup) is a widely used disk encryption solution. By combining LUKS with dm-crypt, you can encrypt your LVM volumes to ensure data remains protected even in case of theft or unauthorized physical access.
- **Encryption procedure:**
  Before creating a logical volume, initialize encryption on the partition or physical volume. Then, create logical volumes inside the encrypted container. This ensures that all stored data is encrypted by default.

Logical Volume Manager (LVM)

Volume Security

2. Access and Permission Management

- **User access control:**
  Ensure that only authorized users have access to LVM volumes. Properly manage permissions on block devices as well as configuration files (e.g., under `/etc/lvm/`).
- **Use sudo and limit privileges:**
  Grant administrative rights only to strictly necessary users, using mechanisms like sudo to minimize exposure in case of compromise.

Logical Volume Manager (LVM)

Volume Security

3. Backup and Snapshots

- **Creating secure snapshots:**
  LVM allows the creation of snapshots to perform consistent backups. Make sure these snapshots are also protected and, if necessary, encrypted to prevent unauthorized access from compromising your backup data.
- **Backup strategy:**
  Establish a regular backup plan and test your recovery procedures. Offsite backups or backups stored on encrypted media enhance overall data security.

Logical Volume Manager (LVM)

Volume Security

4. Updates and Monitoring

- **Regular maintenance:**
  Ensure that the operating system and LVM tools are kept up to date to benefit from the latest security patches.
- **Auditing and monitoring:**
  Set up monitoring and logging tools to detect unusual activity. Regular audits of volume access can help quickly identify any intrusion attempts.

Logical Volume Manager (LVM)

Volume Security

5. Physical and Network Security

- **Securing the hardware:**
  Encryption protects data, but the physical security of servers is equally important. Ensure that your hardware is stored in a physically secure environment.
- **Secure network access:**
  If your LVM volumes are accessible over network services (e.g., for network storage solutions), ensure those services use secure protocols and encrypted connections.

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

BTRFS

BTRFS

Definition

- Btrfs (B-tree File System) is a modern file system designed to offer high flexibility, advanced features, and integrated volume management.
- Created in 2007 and integrated into the Linux kernel in 2009, Btrfs positions itself as an alternative to traditional file systems like ext4 or XFS.

BTRFS

Main Features of Btrfs

1. Integrated Volume Management

- **Adding and removing devices:**
  Btrfs allows you to add or remove disks in a storage pool on the fly, which is very useful to scale according to your needs. For example, with the following command, you can add a new device to an already mounted file system:

```
sudo btrfs device add /dev/sdc /mnt
sudo btrfs balance start /mnt
```

BTRFS

Main Features of Btrfs

1. Integrated Volume Management

- **Built-in software RAID:**
  - Btrfs supports various RAID configurations (such as RAID 0, RAID 1, RAID 10, and experimental modes for RAID 5/6).
  - This means you can configure data redundancy and distribution without additional tools like mdadm or LVM.
  - The following command, for instance, rebalances data across all devices in the pool by applying a RAID 1 strategy for redundancy:

```
sudo btrfs balance start -dconvert=raid1 -mconvert=raid1 /mnt
```

BTRFS

Main Features of Btrfs

2. Subvolumes

- **Nature and behavior:**
  - A subvolume is a logical entity that behaves like an independent file system.
  - It allows you to structure your data more granularly without requiring separate physical partitioning.
  - Each subvolume has its own root inode, making it independently manageable while sharing the global storage space.
- **Use cases:**
  - For example, you can isolate your user data in a `/home` subvolume and system data in another subvolume.
  - This simplifies backups and restorations, as you can create snapshots specific to each subvolume without impacting the whole system.

BTRFS

Main Features of Btrfs

2. Subvolumes

- **Creation and mounting:**
  Creating a subvolume is straightforward:

```
sudo btrfs subvolume create /mnt/home
```

  And to mount a specific subvolume, you can use the `subvol` option in the mount command:

```
sudo mount -o subvol=home /dev/sdb1 /home
```

BTRFS

Main Features of Btrfs

3. Snapshots

- **Fast creation thanks to CoW:**
  The Copy-on-Write mechanism allows snapshots to be created in seconds, as it doesn't require physically copying all the data. The snapshot just marks existing data blocks and only copies blocks that are modified afterward.
- **Read-only vs. read-write snapshots:**
  - *Read-only*: These snapshots ensure that the saved state remains untouched and unmodified — ideal for backups or restore points.
  - *Read-write*: You can also create snapshots that allow modifications. They offer flexibility for testing updates or configurations without affecting the original.

BTRFS

Main Features of Btrfs

3. Snapshots

- **Example of snapshot creation:**

```
sudo btrfs subvolume snapshot -r /mnt/home /mnt/home_snapshot
```

Here, the `-r` flag creates a read-only snapshot. To revert, you can mount this snapshot or even convert it into an active subvolume.

BTRFS

Main Features of Btrfs

4. Copy-on-Write (CoW)

- **Detailed principle:**
  - Instead of writing changes directly to existing blocks, Btrfs allocates new blocks to write the changes.
  - This means that unless CoW is triggered for a write (e.g., when updating a modified file), the original content remains intact.
  - This mechanism enables:
    - Reduced risk of corruption in case of sudden power loss,
    - Easier snapshot creation since the initial content isn't overwritten.

BTRFS

Main Features of Btrfs

4. Copy-on-Write (CoW)

- **Performance impact:**
  - CoW can, in some cases, cause increased fragmentation — especially on heavily modified files.
  - Options like `nodatacow` exist to disable this mechanism for specific files when necessary (e.g., for databases).

BTRFS

Main Features of Btrfs

5. Compression

- **On-the-fly compression:**
  Compression in Btrfs is performed automatically during data writes, without any manual action on the files. You can choose from several algorithms:
  - **zlib**: Good compression, but can be heavier on CPU.
  - **lzo**: Lighter on resources but usually with a lower compression ratio.
  - **zstd**: Offers a good balance between speed and compression efficiency.
- **Usage at mount time:**
  To enable compression, mount the file system with the appropriate option. For example:

```
sudo mount -o compress=zstd /dev/sdb1 /mnt
```

  Once mounted with this option, newly written files will be automatically compressed.

BTRFS

Main Features of Btrfs

6. Summary

- **Volumes and dynamic management** with the ability to add or remove devices and manage RAID natively.
- **Subvolumes** for fine-grained data structuring, simplifying management and backup.
- **Fast snapshots** thanks to the CoW mechanism, offering efficient restore points when needed.
- **Copy-on-Write** ensuring better data integrity and enabling instant copy creation.
- **Real-time compression** to optimize disk space usage while maintaining good performance.

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Boot Sequence

Boot Sequence

Detailed Boot Process

1. Startup via BIOS/UEFI

- **Role:**
  - **BIOS** (Basic Input/Output System) or **UEFI** (Unified Extensible Firmware Interface) initializes the hardware (CPU, RAM, devices).
  - The firmware performs a **POST** (Power-On Self-Test) to check memory and critical components.
  - Then it selects a boot device based on the configured order (hard drive, USB stick, etc.).

Boot Sequence

Detailed Boot Process

2. Bootloader (GRUB)

- **Role:**
  - GRUB (GRand Unified Bootloader) displays a boot menu and allows selection of the kernel to launch.
  - It reads the configuration file (`/boot/grub/grub.cfg`) to locate the kernel (`vmlinuz`) and the initramfs.
  - Thanks to its built-in editor (accessible by pressing `e`), it allows temporary modification of kernel boot arguments.

Boot Sequence

Detailed Boot Process

3. Loading the Linux Kernel and initramfs

- **Linux Kernel:**
  - Loaded into memory (via the `vmlinuz` file), it configures hardware and initializes drivers.
  - The first messages (kernel messages) appear on the console to indicate the hardware detection process.
- **initramfs:**
  - This is a temporary file system loaded into RAM, containing scripts and modules necessary to prepare the mounting of the root file system.
  - Once the root file system is mounted, control is handed over to the user-space environment.

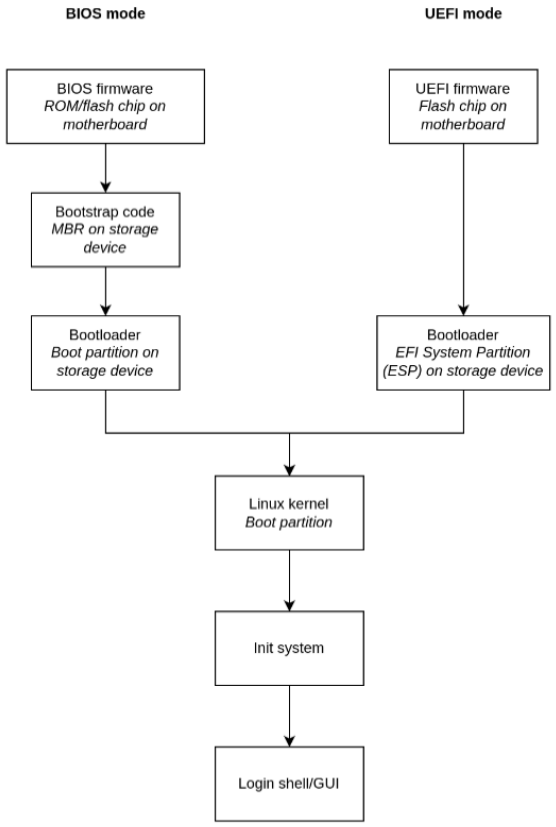
Boot Sequence

Detailed Boot Process

4. Transition to User Space with Init (systemd)

- **Role:**
  - The *init* process (typically **systemd** in modern distributions) takes over as **PID 1**.
  - Systemd mounts the root and other partitions (as defined in `/etc/fstab`), launches services, and sets the default target (often `graphical.target` for desktops or `multi-user.target` for servers).

Boot Sequence



Boot Sequence

Boot Sequence

Passing Boot Arguments

1. Temporary Argument Passing via GRUB

- **Procedure:**
  1. At the GRUB menu, select the boot entry and press `e` to edit.
  2. Locate the line starting with `linux` which already contains parameters (e.g., `ro quiet splash`).
  3. Add the desired argument, for example:
     - **To boot into rescue mode:**

```
systemd.unit=rescue.target
```

     - **To boot into emergency mode:**

```
systemd.unit=emergency.target
```

     - **For direct shell/debug mode:**

```
init=/bin/bash rw
```

  4. Confirm with **Ctrl+X** or **F10** to boot with the new parameters.

Boot Sequence

Permanent Argument Configuration in `/etc/default/grub`

2. Permanent Configuration in `/etc/default/grub`

- **Procedure:**
  1. Edit the `/etc/default/grub` file using a text editor (e.g., `sudo nano /etc/default/grub`).
  2. Modify the `GRUB_CMDLINE_LINUX_DEFAULT` variable to include the desired parameters.
     - Example to force boot into multi-user text mode:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash systemd.unit=multi-user.target"
```

  3. Save the file and regenerate the GRUB configuration with:

```
sudo update-grub
```

     (On some distributions, the command may be `grub-mkconfig -o /boot/grub/grub.cfg`.)

Boot Sequence

Boot Reconstruction

1. Regenerating GRUB Configuration

- **Command:**

```
sudo grub-mkconfig -o /boot/grub/grub.cfg
```

This command detects installed kernels and creates a new configuration file.

2. Rebuilding the initramfs Image

- **On Debian/Ubuntu:**

```
sudo update-initramfs -c -k $(uname -r)
```

- **On Fedora/RedHat:**

```
sudo dracut -f
```

Boot Sequence

Bootloader Reinstallation (If Needed)

3. Reinstalling the Bootloader (if necessary)

- **Procedure from a Live CD/USB:**

  1. Mount the system partition:

```
sudo mount /dev/sda1 /mnt
sudo mount --bind /dev /mnt/dev
sudo chroot /mnt
```

  2. Reinstall GRUB to the disk:

```
grub-install /dev/sda
grub-mkconfig -o /boot/grub/grub.cfg
```

  3. Exit the chroot and reboot.

Boot Sequence

Analyzing System Boot Time

1. Measure Total Boot Time

- **Command:**

```
systemd-analyze
```

- **Example output:**

```
Startup finished in 3.123s (kernel) + 5.456s (initrd) + 10.789s (userspace) = 19.368s
```

  This shows the detailed time taken by the kernel, initramfs, and user space.

2. Identify Slow Services

- **Command:**

```
systemd-analyze blame
```

- **Purpose:**
  - Lists started services sorted by execution time.
  - Helps identify services slowing down the boot process (e.g., `NetworkManager-wait-online.service` might take several seconds).

Boot Sequence

1. Standard Boot and Log Inspection

- **Command to view boot logs:**

```
journalctl -b
```

- **Purpose:**
  - View kernel and systemd messages to verify that all services started correctly.

2. Rescue Mode

- **Procedure:**
  1. In GRUB, edit the command line and add the argument:

```
systemd.unit=rescue.target
```

  2. Confirm to boot into rescue mode, which starts a minimal root shell with only essential services.
- **Usage:**
  - Useful for fixing configurations or correcting errors without the overhead of unnecessary services.

Boot Sequence

3. Emergency Mode

- **Procedure:**
  1. In GRUB, edit the line and add:

     ```
     systemd.unit=emergency.target
     ```

  2. The system will boot into a very minimal shell with the root filesystem mounted read-only.
  3. To make changes, remount the root as read-write:

     ```
     mount -o remount,rw /
     ```

- **Usage:**
  - Ideal for fixing critical errors, such as a broken `/etc/fstab`.

Boot Sequence

4. Debug Mode (init=/bin/bash)

- **Procedure:**
  1. Edit the kernel line in GRUB and add:

     ```
     init=/bin/bash rw
     ```

  2. Boot to get a direct root shell before systemd is launched.
  3. Once inside the shell, you can remount the root as read-write and perform diagnostics:

     ```
     mount -o remount,rw /
     ```

- **Usage:**
  - Provides immediate access to debug the system, fix errors, or manually load modules.

Boot Sequence

5. Root Password Reset

- **Procedure:**
  1. Boot using debug mode:
     - In GRUB, add:

```
init=/bin/bash rw
```

  2. Once the shell is open, remount the root as writable if needed:

```
mount -o remount,rw /
```

  3. Reset the root password:

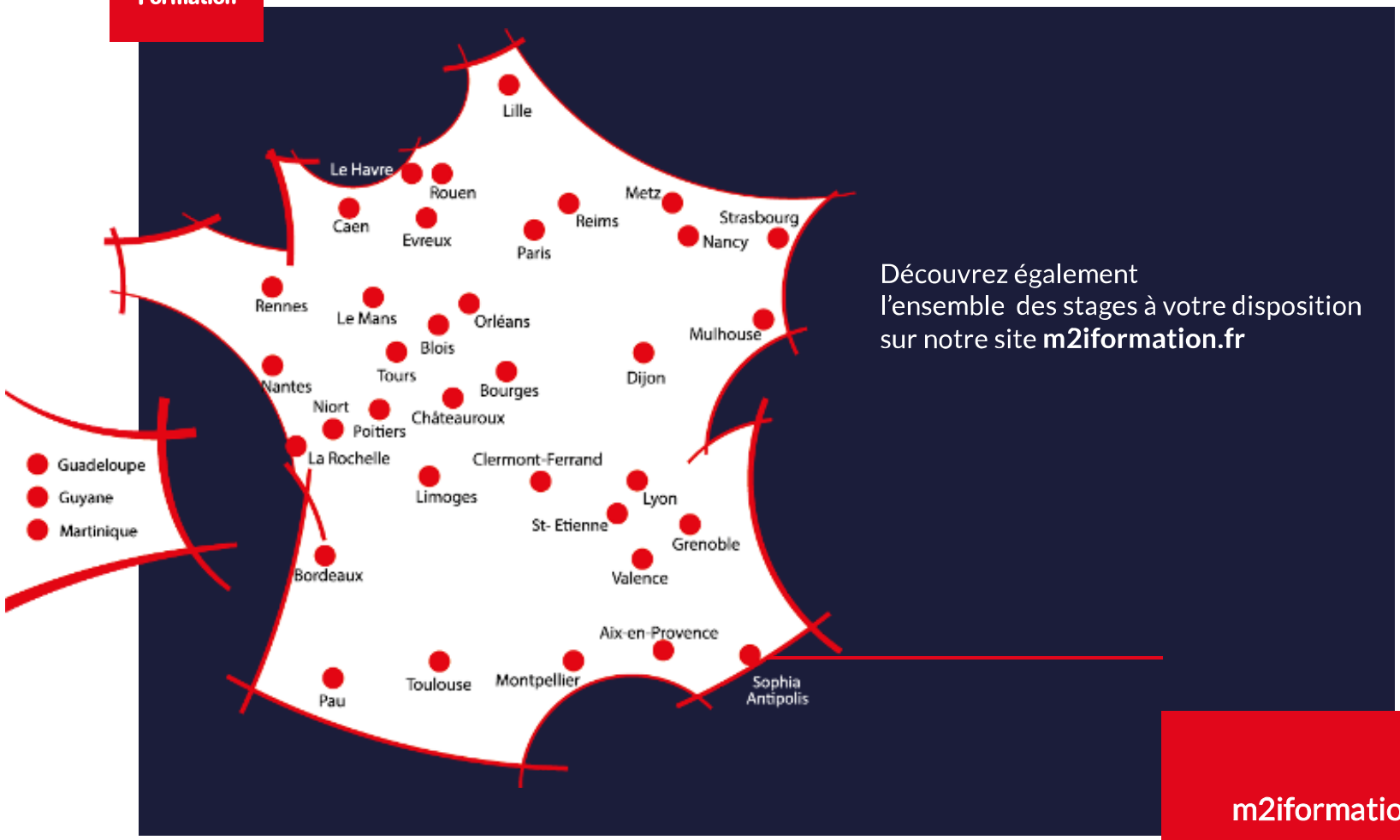
```
passwd root
```

  4. Ensure changes are written to disk by syncing:

```
sync
```

  5. Reboot the system:

```
reboot -f
```



Découvrez également
l'ensemble des stages à votre disposition
sur notre site **m2iformation.fr**

m2iformation.fr